p1

Exported 2/21/2026, 10:10:55 AM

For all problems, the heat rod model is imported from homework 1 (filename heat_rod.py). I'm attaching heat_rod.py code in the Appendix, without the pages assigned for each problem, for clarity.

Similarly, mcmc.py provides the code for the standard Metropolis Hastings algorithm. am_mcmc.py includes the adaptive method for problem 2. gibbs_mcmc.py includes the gibbs sampling step for problem 3. Since these three codes are used for multiple subproblems, the codes are attachedd to the bottom of the pdf, rather than assigned to each specific problem. For all codes/results, please simply view my github repo at: https://github.com/rx9933/ase379l/tree/main/hw6

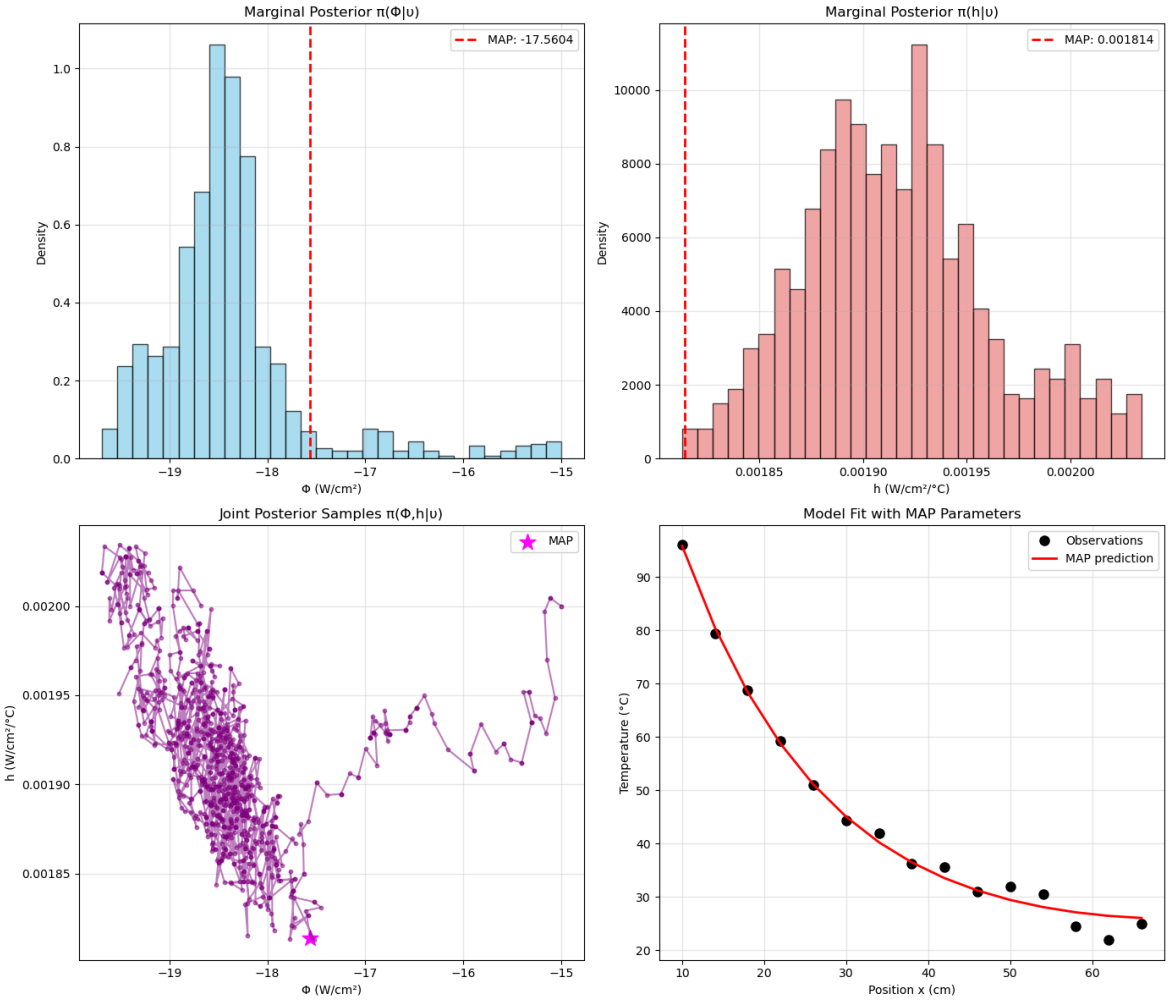
Also: in below plots (on pages 34, 54, and 55), when the histograms are shown in light blue, the KDE is in orange.

1.1

In [4]:

```
import os, sys, numpy as np, matplotlib.pyplot as plt
sys.path.append('../hw1')
import heat_rod as hr
from mcmc import *
from scipy.io import loadmat

data = loadmat("HW06_Problem1.mat")
samples, q_map, posterior_values, acceptance_rate =
run_metropolis_for_heat_equation()

# Save results
np.savez('problem1.1_results.npz',
            samples=samples,
            q_map=q_map,
            posterior_values=posterior_values,
            acceptance_rate=acceptance_rate)

print(f"\nResults saved to 'problem1.1_results.npz'")
```

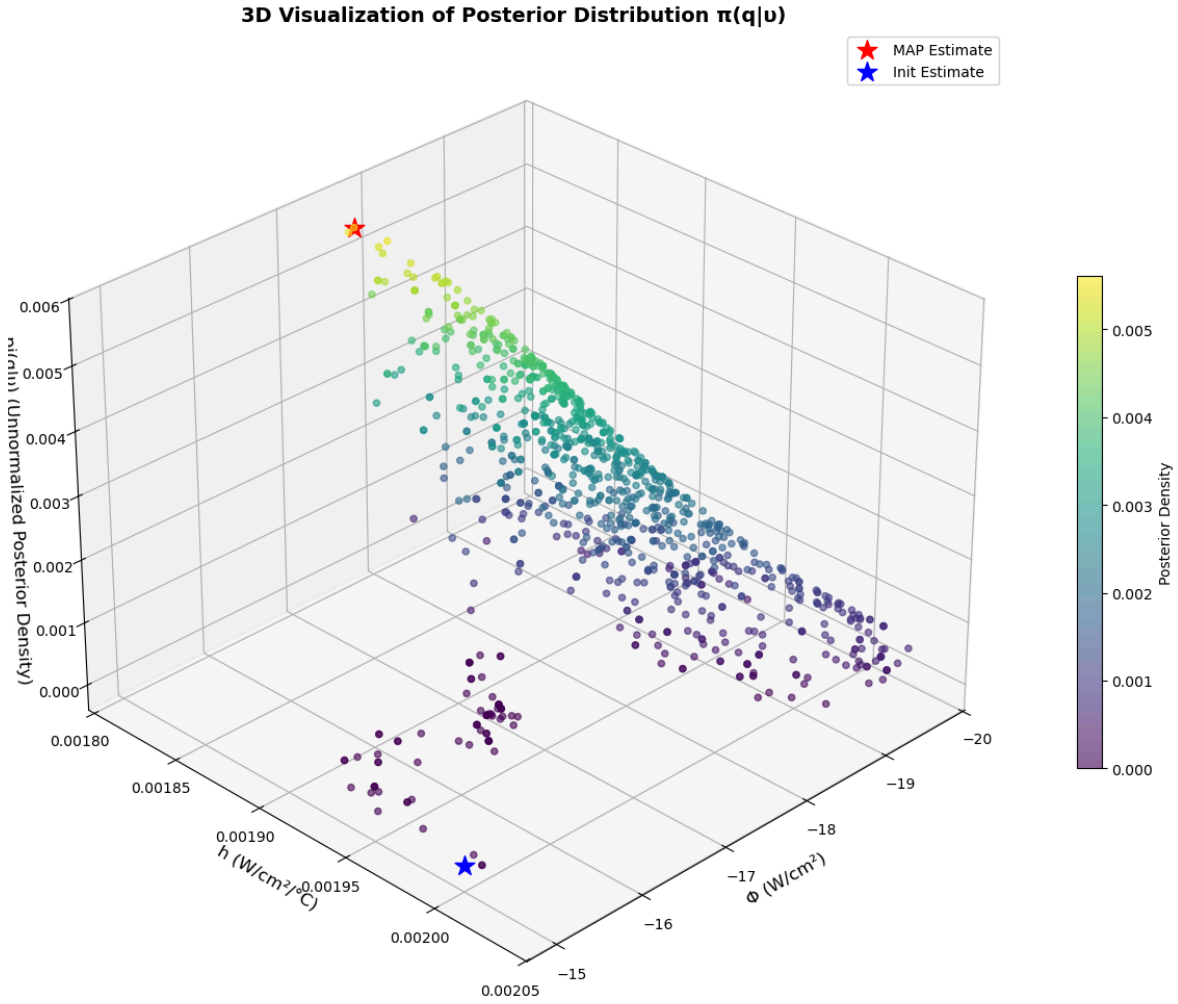
```
Shape of observations: (15,)
Initial test - r(q0, q0) = 1.000000

Running Metropolis algorithm for 1000 iterations...
Proposal distribution: N(q_prev, D) with D = diag([0.01, 2e-10])
Running Metropolis-Hastings MCMC...
  Completed 200/1000 iterations, acceptance rate: 0.719
  Completed 400/1000 iterations, acceptance rate: 0.759
  Completed 600/1000 iterations, acceptance rate: 0.783
  Completed 800/1000 iterations, acceptance rate: 0.796
  Completed 1000/1000 iterations, acceptance rate: 0.799

Final acceptance rate: 0.799

Computing posterior values for MAP estimate...

============================================================
PROBLEM 1.1 RESULTS
============================================================

MAP Estimate (q_MAP):
  Φ = -17.560427 W/cm²
  h = 0.001814 W/cm²/°C
  π(υ|q_MAP)π0(q_MAP) = 5.61e-03

Posterior Distribution π(q|υ) Summary:
============================================================
Φ:
  Mean = -18.395988 W/cm²
  Std  = 0.747602 W/cm²
  95% CI = [-19.464980, -15.928881]

h:
  Mean = 1.916442e-03 W/cm²/°C
  Std  = 4.574893e-05 W/cm²/°C
  95% CI = [1.836325e-03, 2.018715e-03]
```




```
Results saved to 'problem1.1_results.npz'
```

In [5]:

```
fig = plt.figure(figsize=(14, 10))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(samples[:, 0], samples[:, 1], posterior_values,
                     c=posterior_values, cmap='viridis',
                     s=20, alpha=0.6, marker='o')

# Mark the MAP estimate
map_idx = np.argmax(posterior_values)
ax.scatter([samples[map_idx, 0]], [samples[map_idx, 1]],
[posterior_values[map_idx]],
           color='red', s=200, marker='*', label=f'MAP Estimate')

ax.scatter([samples[0, 0]], [samples[0, 1]], [posterior_values[0]],
           color='blue', s=200, marker='*', label=f'Init Estimate')

# Add labels and title
ax.set_xlabel('Φ (W/cm²)', fontsize=12)
ax.set_ylabel('h (W/cm²/°C)', fontsize=12)
ax.set_zlabel('pi(q|υ) (Unnormalized Posterior Density)', fontsize=12)
ax.set_title('3D Visualization of Posterior Distribution π(q|υ)',
fontsize=14, fontweight='bold')

# Add colorbar
cbar = fig.colorbar(scatter, ax=ax, shrink=0.5, aspect=20)
cbar.set_label('Posterior Density', fontsize=10)

# Add legend
ax.legend()

# Set viewing angle for better initial view
ax.view_init(elev=30, azim=45)

plt.tight_layout()
plt.savefig('posterior_3d_scatter.png', dpi=150)
plt.show()
```



In [6]:

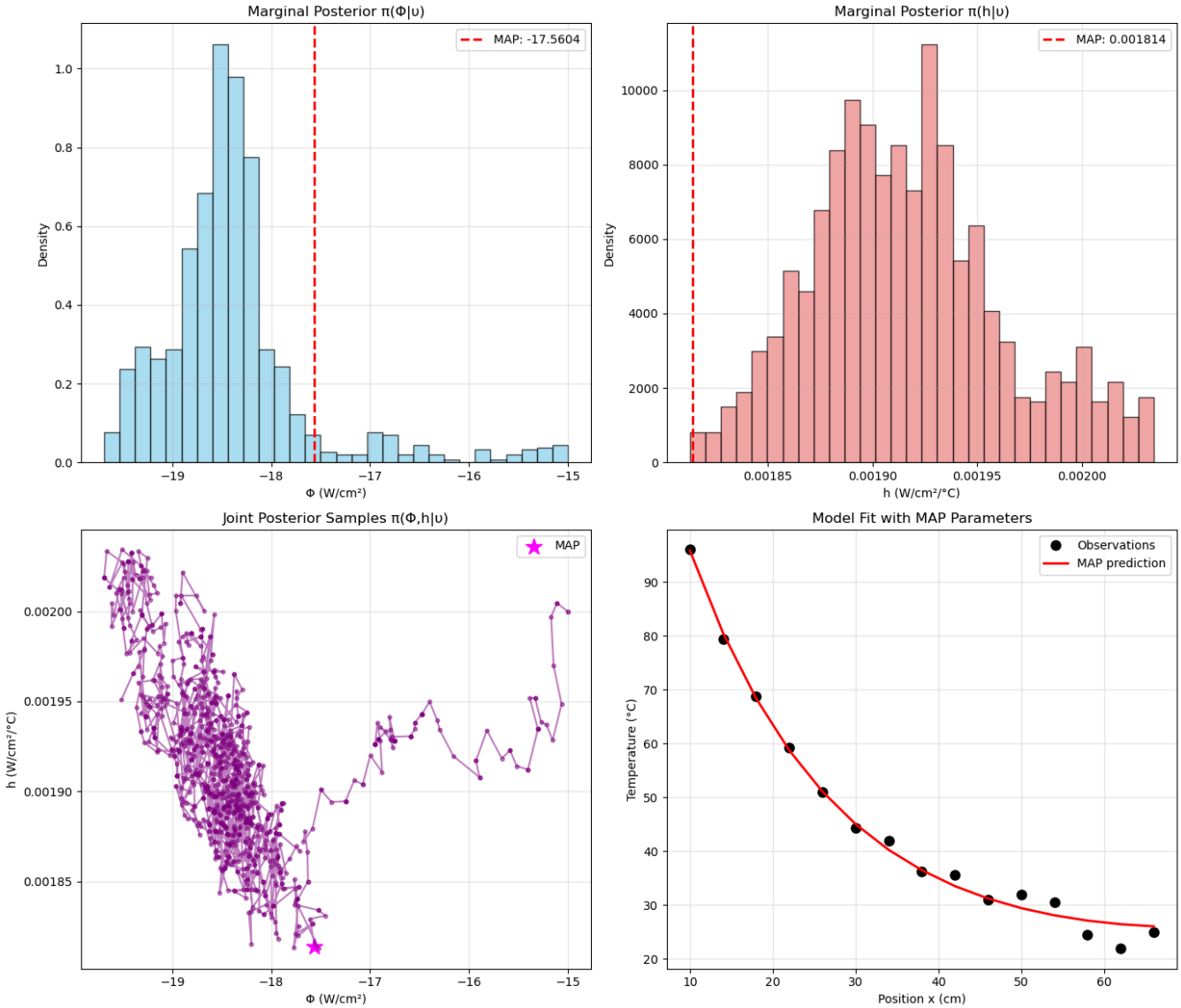
```
max(posterior_values)
```

```
np.float64(0.005609520242829489)
```

[22] $q_{MAP} = [\Phi = -17.56 W/cm^2, h = 0.0018 W/cm^2/°C]$

$\pi(q_{MAP} \mid \nu) \approx 0.0056$

1.2



After burn-in, the chain gradually reaches a stationary distribution, but that does not mean the samples are independent (i.i.d). An MCMC chain satisfies the Markov property,

$$q^{(k+1)} \sim P(, \cdot \mid q^{(k)},),$$

so each sample depends directly on the previous one. Even after convergence to the target distribution,

$$q^{(k)} \sim \pi(q),$$

the samples are identically distributed (after burn-in) but not independent. In particular, there is generally nonzero serial correlation,

$$\mathrm{Cov}!\left(q^{(k)}, q^{(k+1)}\right) \neq 0.$$

Thus, after sufficient burn-in we have draws from the correct stationary distribution $\pi(q)$, but that does not mean $q^{(k)}$ and $q^{(k+1)}$ are independent, identically distributed.

The chain retains autocorrelation because of its sequential construction. This matters because correlated samples reduce the effective sample size (ESS), so $\mathrm{ESS} < N$, and variance estimates must account for autocorrelation. Thinning is can be used to reduce correlation (e.g., save only the r^{th} sample to minimize correlation between samples). Thus, burn-in guarantees convergence to the stationary distribution, but it does not imply independence or i.i.d. samples; MCMC produces correlated draws from the correct (of course approximated) target distribution (eventually).

1.3

Acceptance ratio = 0.799 (near/at 1000/1000 sample)

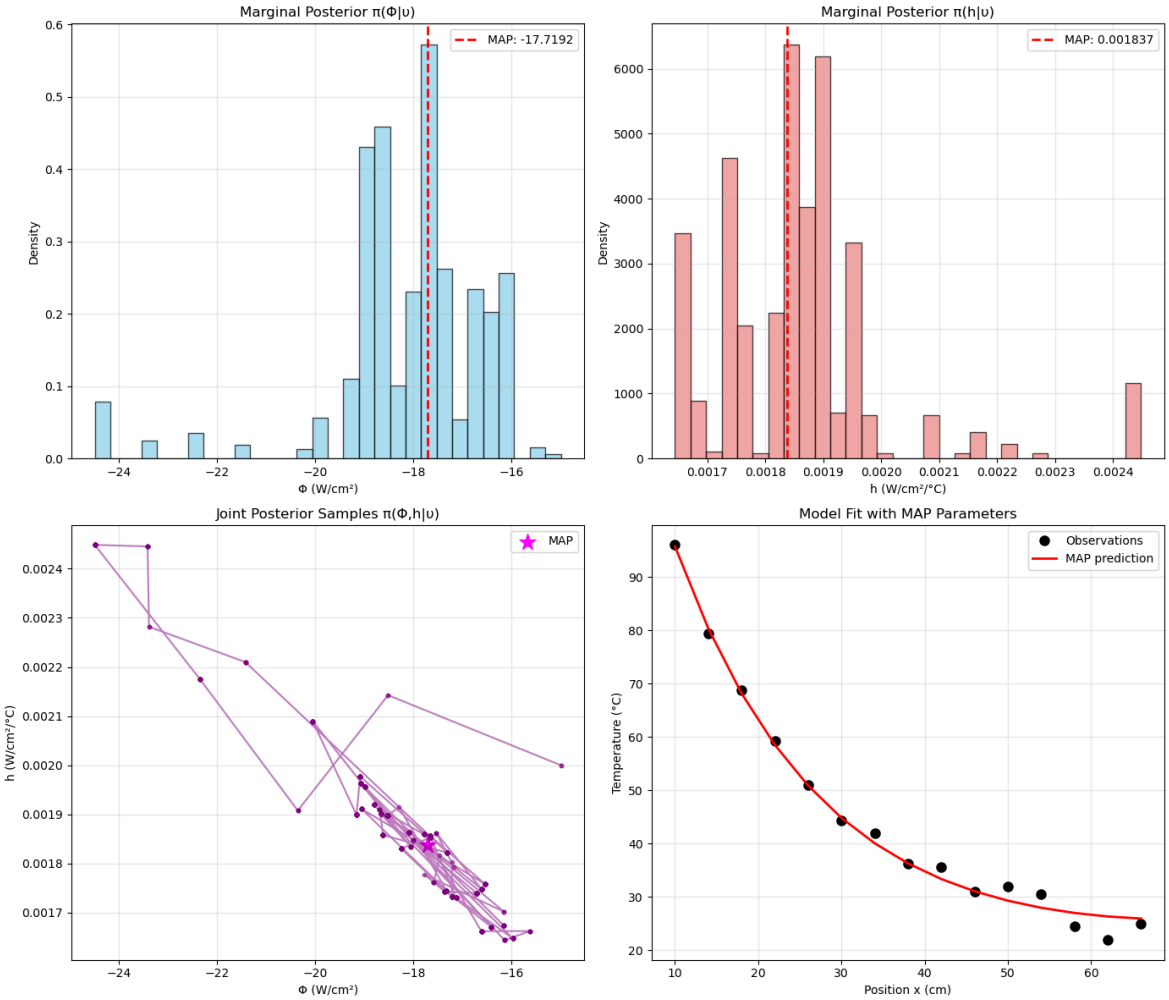
1.4

Large D

In [7]:

```
d2 = np.array([[1e-2, 0],
                    [0, 2e-10]]) * 1000
samples, q_map, posterior_values, acceptance_rate =
run_metropolis_for_heat_equation(D = d2)

# Save results
np.savez('problem1.4_large_D.npz',
            samples=samples,
            q_map=q_map,
            posterior_values=posterior_values,
            acceptance_rate=acceptance_rate)
```

```
Shape of observations: (15,)
Initial test - r(q0, q0) = 1.000000

Running Metropolis algorithm for 1000 iterations...
Proposal distribution: N(q_prev, D) with D = diag([10.0,
2.0000000000000002e-07])
Running Metropolis-Hastings MCMC...
  Completed 200/1000 iterations, acceptance rate: 0.060
  Completed 400/1000 iterations, acceptance rate: 0.063
  Completed 600/1000 iterations, acceptance rate: 0.052
  Completed 800/1000 iterations, acceptance rate: 0.054
  Completed 1000/1000 iterations, acceptance rate: 0.049

Final acceptance rate: 0.049

Computing posterior values for MAP estimate...

============================================================
PROBLEM 1.1 RESULTS
============================================================

MAP Estimate (q_MAP):
  Φ = -17.719167 W/cm²
  h = 0.001837 W/cm²/°C
  π(υ|q_MAP)π0(q_MAP) = 5.03e-03

Posterior Distribution π(q|υ) Summary:
============================================================
Φ:
  Mean = -18.092433 W/cm²
  Std  = 1.612962 W/cm²
  95% CI = [-23.436418, -15.978914]

h:
  Mean = 1.860009e-03 W/cm²/°C
  Std  = 1.514691e-04 W/cm²/°C
  95% CI = [1.647691e-03, 2.445395e-03]
```

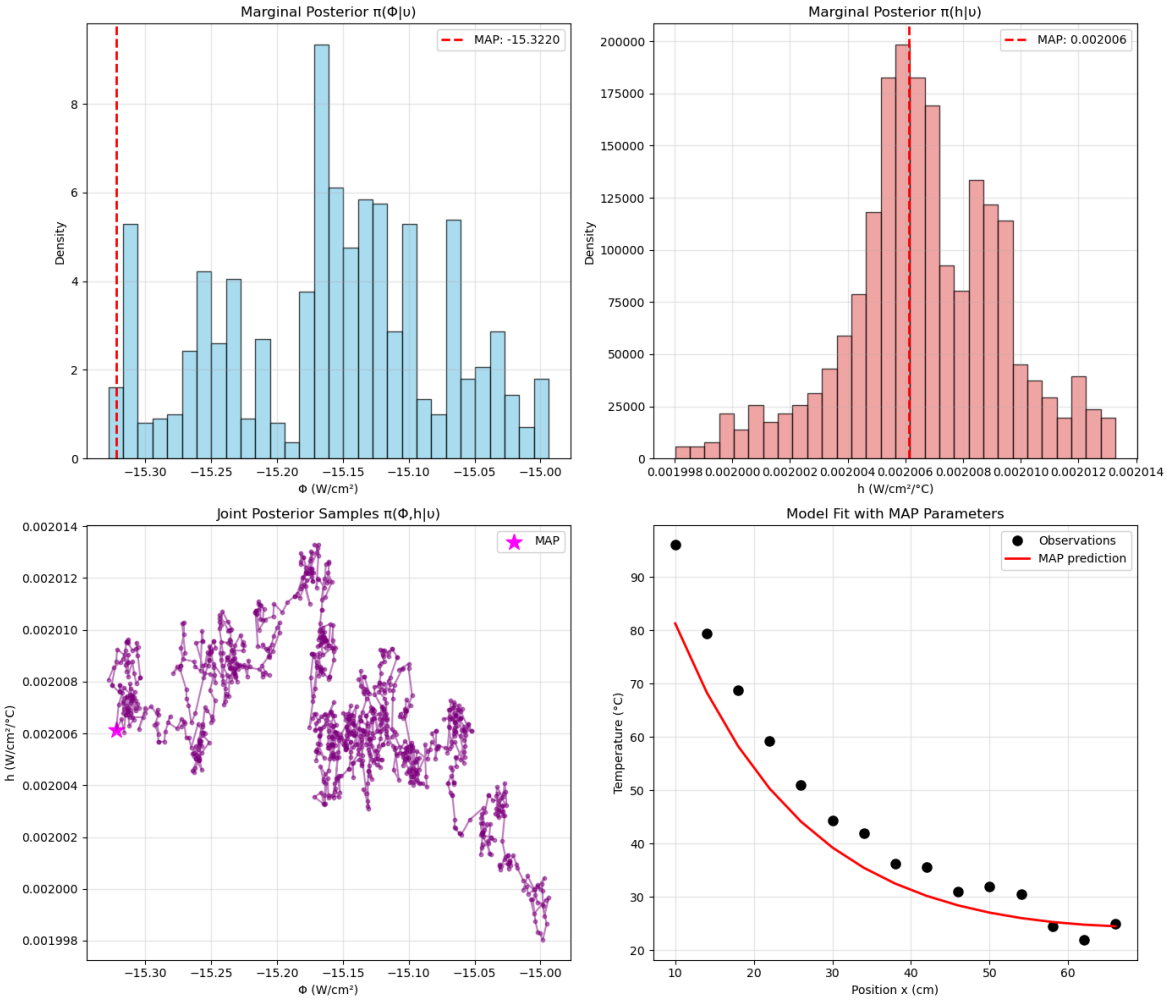


Small D

In [8]:

```
d3 = np.array([[1e-2, 0],
                    [0, 2e-10]]) / 1000
samples, q_map, posterior_values, acceptance_rate =
run_metropolis_for_heat_equation(D = d3)

# Save results
np.savez('problem1.4_small_D.npz',
            samples=samples,
            q_map=q_map,
            posterior_values=posterior_values,
            acceptance_rate=acceptance_rate)
```

```
Shape of observations: (15,)
Initial test - r(q0, q0) = 1.000000

Running Metropolis algorithm for 1000 iterations...
Proposal distribution: N(q_prev, D) with D = diag([1e-05, 2e-13])
Running Metropolis-Hastings MCMC...
  Completed 200/1000 iterations, acceptance rate: 0.905
  Completed 400/1000 iterations, acceptance rate: 0.927
  Completed 600/1000 iterations, acceptance rate: 0.927
  Completed 800/1000 iterations, acceptance rate: 0.935
  Completed 1000/1000 iterations, acceptance rate: 0.932

Final acceptance rate: 0.932

Computing posterior values for MAP estimate...

============================================================
PROBLEM 1.1 RESULTS
============================================================

MAP Estimate (q_MAP):
  Φ = -15.322028 W/cm²
  h = 0.002006 W/cm²/°C
  π(υ|q_MAP)π0(q_MAP) = 6.64e-41

Posterior Distribution π(q|υ) Summary:
============================================================
Φ:
  Mean = -15.159361 W/cm²
  Std  = 0.082691 W/cm²
  95% CI = [-15.315041, -15.010574]

h:
  Mean = 2.006650e-03 W/cm²/°C
  Std  = 2.725994e-06 W/cm²/°C
  95% CI = [2.000325e-03, 2.012206e-03]
```



Off-Diagonal

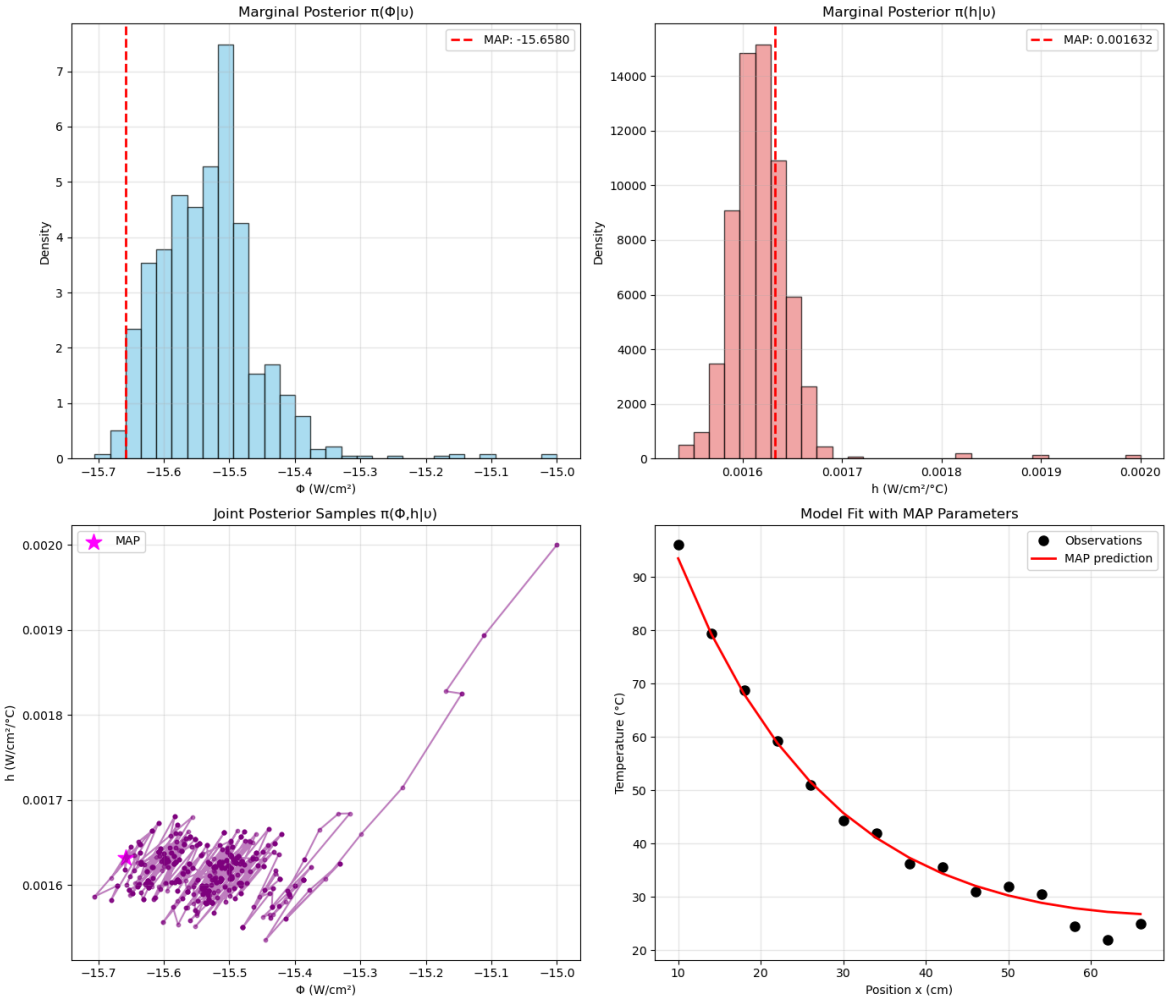
In [9]:

```
d4 = np.array([[1e-2, 1e-5],
                    [0, 2e-10]])
samples, q_map, posterior_values, acceptance_rate =
run_metropolis_for_heat_equation(D = d4)

# Save results
np.savez('problem1.4_off_D.npz',
            samples=samples,
            q_map=q_map,
            posterior_values=posterior_values,
            acceptance_rate=acceptance_rate)
```

```
Shape of observations: (15,)
Initial test - r(q0, q0) = 1.000000

Running Metropolis algorithm for 1000 iterations...
Proposal distribution: N(q_prev, D) with D = diag([0.01, 2e-10])
Running Metropolis-Hastings MCMC...
  Completed 200/1000 iterations, acceptance rate: 0.362
  Completed 400/1000 iterations, acceptance rate: 0.311
  Completed 600/1000 iterations, acceptance rate: 0.294
```

```
/home/mili/ase379l/hw6/mcmc.py:154: RuntimeWarning: covariance is not
symmetric positive-semidefinite.
  return np.random.multivariate_normal(q_current, D)
```

```
  Completed 800/1000 iterations, acceptance rate: 0.293
  Completed 1000/1000 iterations, acceptance rate: 0.284

Final acceptance rate: 0.284

Computing posterior values for MAP estimate...

============================================================
PROBLEM 1.1 RESULTS
============================================================

MAP Estimate (q_MAP):
  Φ = -15.657959 W/cm²
  h = 0.001632 W/cm²/°C
  π(υ|q_MAP)π0(q_MAP) = 1.30e-03

Posterior Distribution π(q|υ) Summary:
============================================================
Φ:
  Mean = -15.533098 W/cm²
  Std  = 0.077003 W/cm²
  95% CI = [-15.651538, -15.386582]

h:
  Mean = 1.618345e-03 W/cm²/°C
  Std  = 3.463688e-05 W/cm²/°C
  95% CI = [1.567229e-03, 1.666204e-03]
```



In manipulating the proposal covariance matrix D and initial guess q_0 for the Metropolis algorithm, we notice that the diagonal elements of D, which control the step sizes for Φ and h respectively, have a direct impact on the acceptance rate and mixing behavior of the chain. When D is too small, the proposal steps become very small, leading to high acceptance rates but extremely poor mixing—the chain explores the parameter space very slowly and exhibits high autocorrelation, requiring many more iterations to adequately sample the posterior distribution. Conversely, when D is too large, the proposals frequently land in regions of extremely low posterior probability, causing the acceptance rate to plummet and the chain to become "stuck" for long periods, again resulting in inefficient exploration. The optimal D balances these extremes, typically yielding acceptance rates between 20-40% and allowing the chain to explore the posterior efficiently. Regarding the initial guess q_0, starting far from the high-probability region (e.g., Φ = -5 or h = 0.01) simply extends the burn-in period as the chain must wander toward the posterior mode, but does not affect the final stationary distribution once convergence is achieved (as long as the extended burn-in is removed).

The use of logarithms in the likelihood and prior calculations effectively prevents numerical underflow issues that could otherwise arise when multiplying many small probability densities, particularly when evaluating proposals far from the posterior mode. This logarithmic approach ensures numerical stability across a wide range of D values and initial conditions, allowing the algorithm to handle poor proposals without encountering floating-point precision limitations.

1.5

In [10]:

```
samples, q_map, posterior_values, acceptance_rate =
run_metropolis_for_heat_equation(M = 100000)

# Save results
np.savez('problem1.5.npz',
            samples=samples,
            q_map=q_map,
            posterior_values=posterior_values,
            acceptance_rate=acceptance_rate)
```

```
Shape of observations: (15,)
Initial test - r(q0, q0) = 1.000000

Running Metropolis algorithm for 100000 iterations...
Proposal distribution: N(q_prev, D) with D = diag([0.01, 2e-10])
Running Metropolis-Hastings MCMC...
  Completed 200/100000 iterations, acceptance rate: 0.719
  Completed 400/100000 iterations, acceptance rate: 0.759
  Completed 600/100000 iterations, acceptance rate: 0.783
  Completed 800/100000 iterations, acceptance rate: 0.796
  Completed 1000/100000 iterations, acceptance rate: 0.799
  Completed 1200/100000 iterations, acceptance rate: 0.808
  Completed 1400/100000 iterations, acceptance rate: 0.816
  Completed 1600/100000 iterations, acceptance rate: 0.821
  Completed 1800/100000 iterations, acceptance rate: 0.827
  Completed 2000/100000 iterations, acceptance rate: 0.822
  Completed 2200/100000 iterations, acceptance rate: 0.822
  Completed 2400/100000 iterations, acceptance rate: 0.821
  Completed 2600/100000 iterations, acceptance rate: 0.823
  Completed 2800/100000 iterations, acceptance rate: 0.818
  Completed 3000/100000 iterations, acceptance rate: 0.818
  Completed 3200/100000 iterations, acceptance rate: 0.819
  Completed 3400/100000 iterations, acceptance rate: 0.819
  Completed 3600/100000 iterations, acceptance rate: 0.818
  Completed 3800/100000 iterations, acceptance rate: 0.815
  Completed 4000/100000 iterations, acceptance rate: 0.813
  Completed 4200/100000 iterations, acceptance rate: 0.814
  Completed 4400/100000 iterations, acceptance rate: 0.813
  Completed 4600/100000 iterations, acceptance rate: 0.813
  Completed 4800/100000 iterations, acceptance rate: 0.813
  Completed 5000/100000 iterations, acceptance rate: 0.813
  Completed 5200/100000 iterations, acceptance rate: 0.814
  Completed 5400/100000 iterations, acceptance rate: 0.813
  Completed 5600/100000 iterations, acceptance rate: 0.813
  Completed 5800/100000 iterations, acceptance rate: 0.813
  Completed 6000/100000 iterations, acceptance rate: 0.813
  Completed 6200/100000 iterations, acceptance rate: 0.811
  Completed 6400/100000 iterations, acceptance rate: 0.811
  Completed 6600/100000 iterations, acceptance rate: 0.812
  Completed 6800/100000 iterations, acceptance rate: 0.812
  Completed 7000/100000 iterations, acceptance rate: 0.812
  Completed 7200/100000 iterations, acceptance rate: 0.811
  Completed 7400/100000 iterations, acceptance rate: 0.809
  Completed 7600/100000 iterations, acceptance rate: 0.809
  Completed 7800/100000 iterations, acceptance rate: 0.809
  Completed 8000/100000 iterations, acceptance rate: 0.807
```

```
Completed 8200/100000 iterations, acceptance rate: 0.807
Completed 8400/100000 iterations, acceptance rate: 0.806
Completed 8600/100000 iterations, acceptance rate: 0.806
Completed 8800/100000 iterations, acceptance rate: 0.807
Completed 9000/100000 iterations, acceptance rate: 0.808
Completed 9200/100000 iterations, acceptance rate: 0.808
Completed 9400/100000 iterations, acceptance rate: 0.807
Completed 9600/100000 iterations, acceptance rate: 0.807
Completed 9800/100000 iterations, acceptance rate: 0.806
Completed 10000/100000 iterations, acceptance rate: 0.805
Completed 10200/100000 iterations, acceptance rate: 0.806
Completed 10400/100000 iterations, acceptance rate: 0.807
Completed 10600/100000 iterations, acceptance rate: 0.807
Completed 10800/100000 iterations, acceptance rate: 0.807
Completed 11000/100000 iterations, acceptance rate: 0.807
Completed 11200/100000 iterations, acceptance rate: 0.808
Completed 11400/100000 iterations, acceptance rate: 0.808
Completed 11600/100000 iterations, acceptance rate: 0.808
Completed 11800/100000 iterations, acceptance rate: 0.809
Completed 12000/100000 iterations, acceptance rate: 0.809
Completed 12200/100000 iterations, acceptance rate: 0.809
Completed 12400/100000 iterations, acceptance rate: 0.809
Completed 12600/100000 iterations, acceptance rate: 0.809
Completed 12800/100000 iterations, acceptance rate: 0.809
Completed 13000/100000 iterations, acceptance rate: 0.809
Completed 13200/100000 iterations, acceptance rate: 0.810
Completed 13400/100000 iterations, acceptance rate: 0.810
Completed 13600/100000 iterations, acceptance rate: 0.811
Completed 13800/100000 iterations, acceptance rate: 0.810
Completed 14000/100000 iterations, acceptance rate: 0.810
Completed 14200/100000 iterations, acceptance rate: 0.810
Completed 14400/100000 iterations, acceptance rate: 0.810
Completed 14600/100000 iterations, acceptance rate: 0.810
Completed 14800/100000 iterations, acceptance rate: 0.810
Completed 15000/100000 iterations, acceptance rate: 0.809
Completed 15200/100000 iterations, acceptance rate: 0.809
Completed 15400/100000 iterations, acceptance rate: 0.809
Completed 15600/100000 iterations, acceptance rate: 0.808
Completed 15800/100000 iterations, acceptance rate: 0.807
Completed 16000/100000 iterations, acceptance rate: 0.807
Completed 16200/100000 iterations, acceptance rate: 0.806
Completed 16400/100000 iterations, acceptance rate: 0.805
Completed 16600/100000 iterations, acceptance rate: 0.805
Completed 16800/100000 iterations, acceptance rate: 0.805
Completed 17000/100000 iterations, acceptance rate: 0.804
Completed 17200/100000 iterations, acceptance rate: 0.804
Completed 17400/100000 iterations, acceptance rate: 0.803
Completed 17600/100000 iterations, acceptance rate: 0.802
```

```
Completed 17800/100000 iterations, acceptance rate: 0.802
Completed 18000/100000 iterations, acceptance rate: 0.802
Completed 18200/100000 iterations, acceptance rate: 0.803
Completed 18400/100000 iterations, acceptance rate: 0.803
Completed 18600/100000 iterations, acceptance rate: 0.803
Completed 18800/100000 iterations, acceptance rate: 0.803
Completed 19000/100000 iterations, acceptance rate: 0.804
Completed 19200/100000 iterations, acceptance rate: 0.804
Completed 19400/100000 iterations, acceptance rate: 0.803
Completed 19600/100000 iterations, acceptance rate: 0.803
Completed 19800/100000 iterations, acceptance rate: 0.802
Completed 20000/100000 iterations, acceptance rate: 0.801
Completed 20200/100000 iterations, acceptance rate: 0.801
Completed 20400/100000 iterations, acceptance rate: 0.802
Completed 20600/100000 iterations, acceptance rate: 0.801
Completed 20800/100000 iterations, acceptance rate: 0.801
Completed 21000/100000 iterations, acceptance rate: 0.801
Completed 21200/100000 iterations, acceptance rate: 0.801
Completed 21400/100000 iterations, acceptance rate: 0.801
Completed 21600/100000 iterations, acceptance rate: 0.801
Completed 21800/100000 iterations, acceptance rate: 0.801
Completed 22000/100000 iterations, acceptance rate: 0.801
Completed 22200/100000 iterations, acceptance rate: 0.801
Completed 22400/100000 iterations, acceptance rate: 0.800
Completed 22600/100000 iterations, acceptance rate: 0.800
Completed 22800/100000 iterations, acceptance rate: 0.800
Completed 23000/100000 iterations, acceptance rate: 0.800
Completed 23200/100000 iterations, acceptance rate: 0.800
Completed 23400/100000 iterations, acceptance rate: 0.800
Completed 23600/100000 iterations, acceptance rate: 0.801
Completed 23800/100000 iterations, acceptance rate: 0.801
Completed 24000/100000 iterations, acceptance rate: 0.801
Completed 24200/100000 iterations, acceptance rate: 0.801
Completed 24400/100000 iterations, acceptance rate: 0.801
Completed 24600/100000 iterations, acceptance rate: 0.801
Completed 24800/100000 iterations, acceptance rate: 0.801
Completed 25000/100000 iterations, acceptance rate: 0.801
Completed 25200/100000 iterations, acceptance rate: 0.801
Completed 25400/100000 iterations, acceptance rate: 0.802
Completed 25600/100000 iterations, acceptance rate: 0.802
Completed 25800/100000 iterations, acceptance rate: 0.802
Completed 26000/100000 iterations, acceptance rate: 0.802
Completed 26200/100000 iterations, acceptance rate: 0.802
Completed 26400/100000 iterations, acceptance rate: 0.801
Completed 26600/100000 iterations, acceptance rate: 0.802
Completed 26800/100000 iterations, acceptance rate: 0.802
Completed 27000/100000 iterations, acceptance rate: 0.802
Completed 27200/100000 iterations, acceptance rate: 0.802
```

```
Completed 27400/100000 iterations, acceptance rate: 0.802
Completed 27600/100000 iterations, acceptance rate: 0.802
Completed 27800/100000 iterations, acceptance rate: 0.802
Completed 28000/100000 iterations, acceptance rate: 0.802
Completed 28200/100000 iterations, acceptance rate: 0.802
Completed 28400/100000 iterations, acceptance rate: 0.802
Completed 28600/100000 iterations, acceptance rate: 0.802
Completed 28800/100000 iterations, acceptance rate: 0.802
Completed 29000/100000 iterations, acceptance rate: 0.802
Completed 29200/100000 iterations, acceptance rate: 0.803
Completed 29400/100000 iterations, acceptance rate: 0.803
Completed 29600/100000 iterations, acceptance rate: 0.803
Completed 29800/100000 iterations, acceptance rate: 0.803
Completed 30000/100000 iterations, acceptance rate: 0.802
Completed 30200/100000 iterations, acceptance rate: 0.802
Completed 30400/100000 iterations, acceptance rate: 0.802
Completed 30600/100000 iterations, acceptance rate: 0.803
Completed 30800/100000 iterations, acceptance rate: 0.803
Completed 31000/100000 iterations, acceptance rate: 0.803
Completed 31200/100000 iterations, acceptance rate: 0.803
Completed 31400/100000 iterations, acceptance rate: 0.803
Completed 31600/100000 iterations, acceptance rate: 0.803
Completed 31800/100000 iterations, acceptance rate: 0.803
Completed 32000/100000 iterations, acceptance rate: 0.803
Completed 32200/100000 iterations, acceptance rate: 0.803
Completed 32400/100000 iterations, acceptance rate: 0.803
Completed 32600/100000 iterations, acceptance rate: 0.803
Completed 32800/100000 iterations, acceptance rate: 0.803
Completed 33000/100000 iterations, acceptance rate: 0.803
Completed 33200/100000 iterations, acceptance rate: 0.803
Completed 33400/100000 iterations, acceptance rate: 0.803
Completed 33600/100000 iterations, acceptance rate: 0.803
Completed 33800/100000 iterations, acceptance rate: 0.803
Completed 34000/100000 iterations, acceptance rate: 0.803
Completed 34200/100000 iterations, acceptance rate: 0.803
Completed 34400/100000 iterations, acceptance rate: 0.803
Completed 34600/100000 iterations, acceptance rate: 0.804
Completed 34800/100000 iterations, acceptance rate: 0.803
Completed 35000/100000 iterations, acceptance rate: 0.804
Completed 35200/100000 iterations, acceptance rate: 0.804
Completed 35400/100000 iterations, acceptance rate: 0.804
Completed 35600/100000 iterations, acceptance rate: 0.804
Completed 35800/100000 iterations, acceptance rate: 0.804
Completed 36000/100000 iterations, acceptance rate: 0.804
Completed 36200/100000 iterations, acceptance rate: 0.804
Completed 36400/100000 iterations, acceptance rate: 0.803
Completed 36600/100000 iterations, acceptance rate: 0.803
Completed 36800/100000 iterations, acceptance rate: 0.804
```

```
Completed 37000/100000 iterations, acceptance rate: 0.804
Completed 37200/100000 iterations, acceptance rate: 0.804
Completed 37400/100000 iterations, acceptance rate: 0.803
Completed 37600/100000 iterations, acceptance rate: 0.803
Completed 37800/100000 iterations, acceptance rate: 0.803
Completed 38000/100000 iterations, acceptance rate: 0.803
Completed 38200/100000 iterations, acceptance rate: 0.803
Completed 38400/100000 iterations, acceptance rate: 0.803
Completed 38600/100000 iterations, acceptance rate: 0.803
Completed 38800/100000 iterations, acceptance rate: 0.803
Completed 39000/100000 iterations, acceptance rate: 0.803
Completed 39200/100000 iterations, acceptance rate: 0.803
Completed 39400/100000 iterations, acceptance rate: 0.803
Completed 39600/100000 iterations, acceptance rate: 0.803
Completed 39800/100000 iterations, acceptance rate: 0.803
Completed 40000/100000 iterations, acceptance rate: 0.803
Completed 40200/100000 iterations, acceptance rate: 0.803
Completed 40400/100000 iterations, acceptance rate: 0.803
Completed 40600/100000 iterations, acceptance rate: 0.803
Completed 40800/100000 iterations, acceptance rate: 0.803
Completed 41000/100000 iterations, acceptance rate: 0.803
Completed 41200/100000 iterations, acceptance rate: 0.803
Completed 41400/100000 iterations, acceptance rate: 0.803
Completed 41600/100000 iterations, acceptance rate: 0.803
Completed 41800/100000 iterations, acceptance rate: 0.804
Completed 42000/100000 iterations, acceptance rate: 0.804
Completed 42200/100000 iterations, acceptance rate: 0.804
Completed 42400/100000 iterations, acceptance rate: 0.804
Completed 42600/100000 iterations, acceptance rate: 0.804
Completed 42800/100000 iterations, acceptance rate: 0.804
Completed 43000/100000 iterations, acceptance rate: 0.804
Completed 43200/100000 iterations, acceptance rate: 0.804
Completed 43400/100000 iterations, acceptance rate: 0.804
Completed 43600/100000 iterations, acceptance rate: 0.804
Completed 43800/100000 iterations, acceptance rate: 0.804
Completed 44000/100000 iterations, acceptance rate: 0.804
Completed 44200/100000 iterations, acceptance rate: 0.805
Completed 44400/100000 iterations, acceptance rate: 0.805
Completed 44600/100000 iterations, acceptance rate: 0.805
Completed 44800/100000 iterations, acceptance rate: 0.805
Completed 45000/100000 iterations, acceptance rate: 0.805
Completed 45200/100000 iterations, acceptance rate: 0.805
Completed 45400/100000 iterations, acceptance rate: 0.805
Completed 45600/100000 iterations, acceptance rate: 0.805
Completed 45800/100000 iterations, acceptance rate: 0.805
Completed 46000/100000 iterations, acceptance rate: 0.804
Completed 46200/100000 iterations, acceptance rate: 0.804
Completed 46400/100000 iterations, acceptance rate: 0.804
```

```
Completed 46600/100000 iterations, acceptance rate: 0.804
Completed 46800/100000 iterations, acceptance rate: 0.804
Completed 47000/100000 iterations, acceptance rate: 0.804
Completed 47200/100000 iterations, acceptance rate: 0.804
Completed 47400/100000 iterations, acceptance rate: 0.804
Completed 47600/100000 iterations, acceptance rate: 0.804
Completed 47800/100000 iterations, acceptance rate: 0.804
Completed 48000/100000 iterations, acceptance rate: 0.804
Completed 48200/100000 iterations, acceptance rate: 0.804
Completed 48400/100000 iterations, acceptance rate: 0.804
Completed 48600/100000 iterations, acceptance rate: 0.804
Completed 48800/100000 iterations, acceptance rate: 0.804
Completed 49000/100000 iterations, acceptance rate: 0.804
Completed 49200/100000 iterations, acceptance rate: 0.804
Completed 49400/100000 iterations, acceptance rate: 0.804
Completed 49600/100000 iterations, acceptance rate: 0.804
Completed 49800/100000 iterations, acceptance rate: 0.804
Completed 50000/100000 iterations, acceptance rate: 0.804
Completed 50200/100000 iterations, acceptance rate: 0.804
Completed 50400/100000 iterations, acceptance rate: 0.804
Completed 50600/100000 iterations, acceptance rate: 0.804
Completed 50800/100000 iterations, acceptance rate: 0.804
Completed 51000/100000 iterations, acceptance rate: 0.804
Completed 51200/100000 iterations, acceptance rate: 0.804
Completed 51400/100000 iterations, acceptance rate: 0.804
Completed 51600/100000 iterations, acceptance rate: 0.804
Completed 51800/100000 iterations, acceptance rate: 0.804
Completed 52000/100000 iterations, acceptance rate: 0.804
Completed 52200/100000 iterations, acceptance rate: 0.804
Completed 52400/100000 iterations, acceptance rate: 0.804
Completed 52600/100000 iterations, acceptance rate: 0.804
Completed 52800/100000 iterations, acceptance rate: 0.804
Completed 53000/100000 iterations, acceptance rate: 0.804
Completed 53200/100000 iterations, acceptance rate: 0.804
Completed 53400/100000 iterations, acceptance rate: 0.804
Completed 53600/100000 iterations, acceptance rate: 0.804
Completed 53800/100000 iterations, acceptance rate: 0.804
Completed 54000/100000 iterations, acceptance rate: 0.804
Completed 54200/100000 iterations, acceptance rate: 0.804
Completed 54400/100000 iterations, acceptance rate: 0.803
Completed 54600/100000 iterations, acceptance rate: 0.803
Completed 54800/100000 iterations, acceptance rate: 0.803
Completed 55000/100000 iterations, acceptance rate: 0.804
Completed 55200/100000 iterations, acceptance rate: 0.804
Completed 55400/100000 iterations, acceptance rate: 0.803
Completed 55600/100000 iterations, acceptance rate: 0.804
Completed 55800/100000 iterations, acceptance rate: 0.804
Completed 56000/100000 iterations, acceptance rate: 0.804
```

```
Completed 56200/100000 iterations, acceptance rate: 0.804
Completed 56400/100000 iterations, acceptance rate: 0.804
Completed 56600/100000 iterations, acceptance rate: 0.804
Completed 56800/100000 iterations, acceptance rate: 0.804
Completed 57000/100000 iterations, acceptance rate: 0.804
Completed 57200/100000 iterations, acceptance rate: 0.804
Completed 57400/100000 iterations, acceptance rate: 0.804
Completed 57600/100000 iterations, acceptance rate: 0.804
Completed 57800/100000 iterations, acceptance rate: 0.804
Completed 58000/100000 iterations, acceptance rate: 0.804
Completed 58200/100000 iterations, acceptance rate: 0.804
Completed 58400/100000 iterations, acceptance rate: 0.804
Completed 58600/100000 iterations, acceptance rate: 0.804
Completed 58800/100000 iterations, acceptance rate: 0.804
Completed 59000/100000 iterations, acceptance rate: 0.804
Completed 59200/100000 iterations, acceptance rate: 0.804
Completed 59400/100000 iterations, acceptance rate: 0.804
Completed 59600/100000 iterations, acceptance rate: 0.804
Completed 59800/100000 iterations, acceptance rate: 0.804
Completed 60000/100000 iterations, acceptance rate: 0.804
Completed 60200/100000 iterations, acceptance rate: 0.804
Completed 60400/100000 iterations, acceptance rate: 0.804
Completed 60600/100000 iterations, acceptance rate: 0.804
Completed 60800/100000 iterations, acceptance rate: 0.804
Completed 61000/100000 iterations, acceptance rate: 0.804
Completed 61200/100000 iterations, acceptance rate: 0.804
Completed 61400/100000 iterations, acceptance rate: 0.804
Completed 61600/100000 iterations, acceptance rate: 0.804
Completed 61800/100000 iterations, acceptance rate: 0.804
Completed 62000/100000 iterations, acceptance rate: 0.804
Completed 62200/100000 iterations, acceptance rate: 0.804
Completed 62400/100000 iterations, acceptance rate: 0.804
Completed 62600/100000 iterations, acceptance rate: 0.804
Completed 62800/100000 iterations, acceptance rate: 0.804
Completed 63000/100000 iterations, acceptance rate: 0.804
Completed 63200/100000 iterations, acceptance rate: 0.804
Completed 63400/100000 iterations, acceptance rate: 0.804
Completed 63600/100000 iterations, acceptance rate: 0.804
Completed 63800/100000 iterations, acceptance rate: 0.804
Completed 64000/100000 iterations, acceptance rate: 0.804
Completed 64200/100000 iterations, acceptance rate: 0.804
Completed 64400/100000 iterations, acceptance rate: 0.804
Completed 64600/100000 iterations, acceptance rate: 0.804
Completed 64800/100000 iterations, acceptance rate: 0.804
Completed 65000/100000 iterations, acceptance rate: 0.804
Completed 65200/100000 iterations, acceptance rate: 0.804
Completed 65400/100000 iterations, acceptance rate: 0.804
Completed 65600/100000 iterations, acceptance rate: 0.804
```

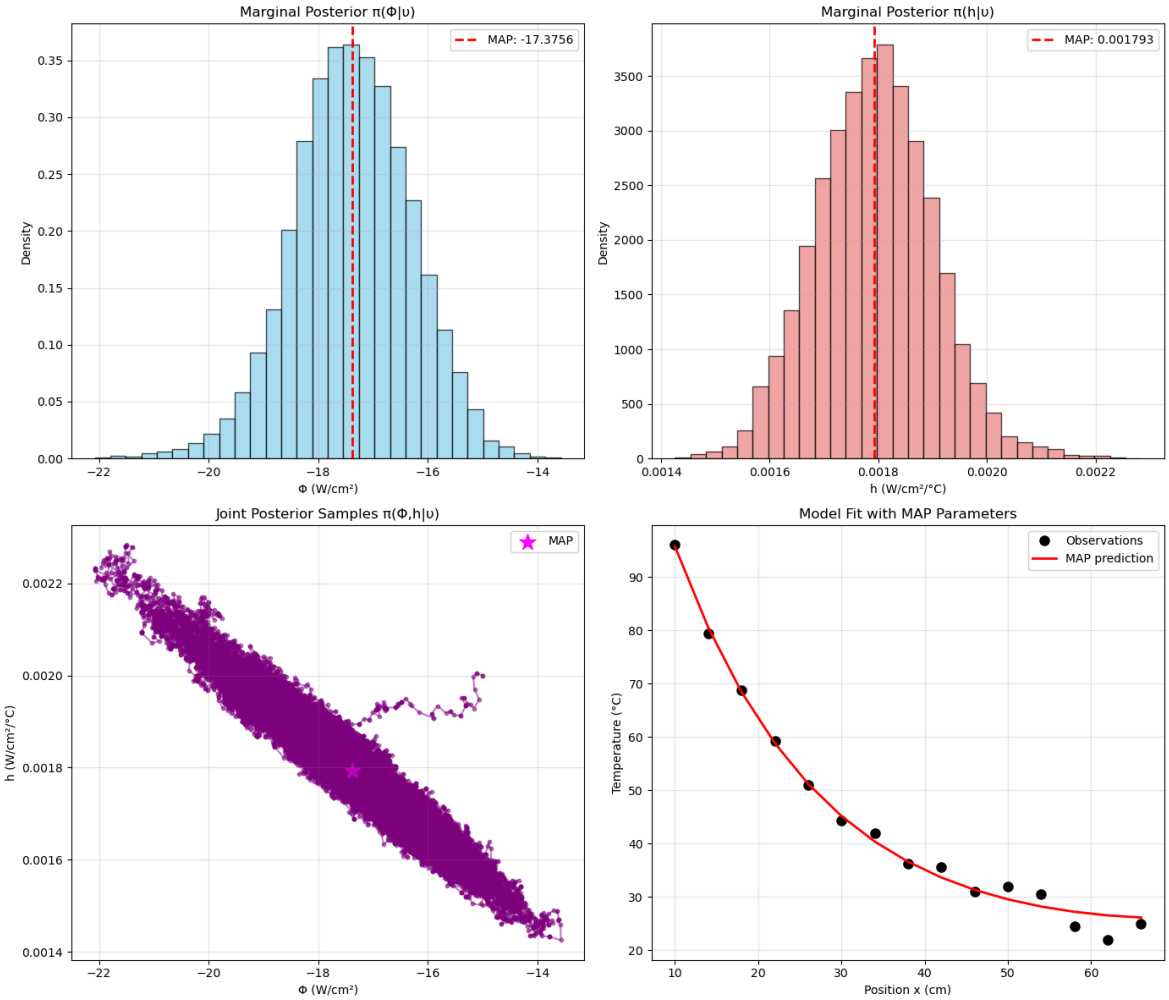
```
Completed 65800/100000 iterations, acceptance rate: 0.804
Completed 66000/100000 iterations, acceptance rate: 0.804
Completed 66200/100000 iterations, acceptance rate: 0.804
Completed 66400/100000 iterations, acceptance rate: 0.804
Completed 66600/100000 iterations, acceptance rate: 0.804
Completed 66800/100000 iterations, acceptance rate: 0.804
Completed 67000/100000 iterations, acceptance rate: 0.805
Completed 67200/100000 iterations, acceptance rate: 0.805
Completed 67400/100000 iterations, acceptance rate: 0.805
Completed 67600/100000 iterations, acceptance rate: 0.805
Completed 67800/100000 iterations, acceptance rate: 0.805
Completed 68000/100000 iterations, acceptance rate: 0.805
Completed 68200/100000 iterations, acceptance rate: 0.805
Completed 68400/100000 iterations, acceptance rate: 0.804
Completed 68600/100000 iterations, acceptance rate: 0.804
Completed 68800/100000 iterations, acceptance rate: 0.805
Completed 69000/100000 iterations, acceptance rate: 0.805
Completed 69200/100000 iterations, acceptance rate: 0.805
Completed 69400/100000 iterations, acceptance rate: 0.804
Completed 69600/100000 iterations, acceptance rate: 0.804
Completed 69800/100000 iterations, acceptance rate: 0.804
Completed 70000/100000 iterations, acceptance rate: 0.804
Completed 70200/100000 iterations, acceptance rate: 0.804
Completed 70400/100000 iterations, acceptance rate: 0.804
Completed 70600/100000 iterations, acceptance rate: 0.804
Completed 70800/100000 iterations, acceptance rate: 0.804
Completed 71000/100000 iterations, acceptance rate: 0.804
Completed 71200/100000 iterations, acceptance rate: 0.804
Completed 71400/100000 iterations, acceptance rate: 0.804
Completed 71600/100000 iterations, acceptance rate: 0.804
Completed 71800/100000 iterations, acceptance rate: 0.804
Completed 72000/100000 iterations, acceptance rate: 0.804
Completed 72200/100000 iterations, acceptance rate: 0.804
Completed 72400/100000 iterations, acceptance rate: 0.804
Completed 72600/100000 iterations, acceptance rate: 0.804
Completed 72800/100000 iterations, acceptance rate: 0.804
Completed 73000/100000 iterations, acceptance rate: 0.804
Completed 73200/100000 iterations, acceptance rate: 0.804
Completed 73400/100000 iterations, acceptance rate: 0.804
Completed 73600/100000 iterations, acceptance rate: 0.804
Completed 73800/100000 iterations, acceptance rate: 0.804
Completed 74000/100000 iterations, acceptance rate: 0.804
Completed 74200/100000 iterations, acceptance rate: 0.804
Completed 74400/100000 iterations, acceptance rate: 0.804
Completed 74600/100000 iterations, acceptance rate: 0.804
Completed 74800/100000 iterations, acceptance rate: 0.804
Completed 75000/100000 iterations, acceptance rate: 0.804
Completed 75200/100000 iterations, acceptance rate: 0.804
```

```
Completed 75400/100000 iterations, acceptance rate: 0.804
Completed 75600/100000 iterations, acceptance rate: 0.804
Completed 75800/100000 iterations, acceptance rate: 0.804
Completed 76000/100000 iterations, acceptance rate: 0.804
Completed 76200/100000 iterations, acceptance rate: 0.804
Completed 76400/100000 iterations, acceptance rate: 0.804
Completed 76600/100000 iterations, acceptance rate: 0.804
Completed 76800/100000 iterations, acceptance rate: 0.804
Completed 77000/100000 iterations, acceptance rate: 0.804
Completed 77200/100000 iterations, acceptance rate: 0.804
Completed 77400/100000 iterations, acceptance rate: 0.804
Completed 77600/100000 iterations, acceptance rate: 0.804
Completed 77800/100000 iterations, acceptance rate: 0.804
Completed 78000/100000 iterations, acceptance rate: 0.804
Completed 78200/100000 iterations, acceptance rate: 0.804
Completed 78400/100000 iterations, acceptance rate: 0.804
Completed 78600/100000 iterations, acceptance rate: 0.804
Completed 78800/100000 iterations, acceptance rate: 0.804
Completed 79000/100000 iterations, acceptance rate: 0.804
Completed 79200/100000 iterations, acceptance rate: 0.804
Completed 79400/100000 iterations, acceptance rate: 0.804
Completed 79600/100000 iterations, acceptance rate: 0.803
Completed 79800/100000 iterations, acceptance rate: 0.804
Completed 80000/100000 iterations, acceptance rate: 0.804
Completed 80200/100000 iterations, acceptance rate: 0.804
Completed 80400/100000 iterations, acceptance rate: 0.804
Completed 80600/100000 iterations, acceptance rate: 0.803
Completed 80800/100000 iterations, acceptance rate: 0.803
Completed 81000/100000 iterations, acceptance rate: 0.803
Completed 81200/100000 iterations, acceptance rate: 0.803
Completed 81400/100000 iterations, acceptance rate: 0.803
Completed 81600/100000 iterations, acceptance rate: 0.803
Completed 81800/100000 iterations, acceptance rate: 0.804
Completed 82000/100000 iterations, acceptance rate: 0.804
Completed 82200/100000 iterations, acceptance rate: 0.804
Completed 82400/100000 iterations, acceptance rate: 0.804
Completed 82600/100000 iterations, acceptance rate: 0.804
Completed 82800/100000 iterations, acceptance rate: 0.804
Completed 83000/100000 iterations, acceptance rate: 0.804
Completed 83200/100000 iterations, acceptance rate: 0.804
Completed 83400/100000 iterations, acceptance rate: 0.804
Completed 83600/100000 iterations, acceptance rate: 0.804
Completed 83800/100000 iterations, acceptance rate: 0.804
Completed 84000/100000 iterations, acceptance rate: 0.804
Completed 84200/100000 iterations, acceptance rate: 0.804
Completed 84400/100000 iterations, acceptance rate: 0.804
Completed 84600/100000 iterations, acceptance rate: 0.804
Completed 84800/100000 iterations, acceptance rate: 0.804
```

```
Completed 85000/100000 iterations, acceptance rate: 0.804
Completed 85200/100000 iterations, acceptance rate: 0.804
Completed 85400/100000 iterations, acceptance rate: 0.804
Completed 85600/100000 iterations, acceptance rate: 0.804
Completed 85800/100000 iterations, acceptance rate: 0.804
Completed 86000/100000 iterations, acceptance rate: 0.804
Completed 86200/100000 iterations, acceptance rate: 0.804
Completed 86400/100000 iterations, acceptance rate: 0.804
Completed 86600/100000 iterations, acceptance rate: 0.804
Completed 86800/100000 iterations, acceptance rate: 0.804
Completed 87000/100000 iterations, acceptance rate: 0.804
Completed 87200/100000 iterations, acceptance rate: 0.804
Completed 87400/100000 iterations, acceptance rate: 0.804
Completed 87600/100000 iterations, acceptance rate: 0.804
Completed 87800/100000 iterations, acceptance rate: 0.804
Completed 88000/100000 iterations, acceptance rate: 0.804
Completed 88200/100000 iterations, acceptance rate: 0.804
Completed 88400/100000 iterations, acceptance rate: 0.804
Completed 88600/100000 iterations, acceptance rate: 0.803
Completed 88800/100000 iterations, acceptance rate: 0.803
Completed 89000/100000 iterations, acceptance rate: 0.803
Completed 89200/100000 iterations, acceptance rate: 0.804
Completed 89400/100000 iterations, acceptance rate: 0.803
Completed 89600/100000 iterations, acceptance rate: 0.803
Completed 89800/100000 iterations, acceptance rate: 0.803
Completed 90000/100000 iterations, acceptance rate: 0.803
Completed 90200/100000 iterations, acceptance rate: 0.803
Completed 90400/100000 iterations, acceptance rate: 0.803
Completed 90600/100000 iterations, acceptance rate: 0.803
Completed 90800/100000 iterations, acceptance rate: 0.804
Completed 91000/100000 iterations, acceptance rate: 0.803
Completed 91200/100000 iterations, acceptance rate: 0.803
Completed 91400/100000 iterations, acceptance rate: 0.803
Completed 91600/100000 iterations, acceptance rate: 0.804
Completed 91800/100000 iterations, acceptance rate: 0.804
Completed 92000/100000 iterations, acceptance rate: 0.804
Completed 92200/100000 iterations, acceptance rate: 0.804
Completed 92400/100000 iterations, acceptance rate: 0.804
Completed 92600/100000 iterations, acceptance rate: 0.804
Completed 92800/100000 iterations, acceptance rate: 0.804
Completed 93000/100000 iterations, acceptance rate: 0.804
Completed 93200/100000 iterations, acceptance rate: 0.804
Completed 93400/100000 iterations, acceptance rate: 0.804
Completed 93600/100000 iterations, acceptance rate: 0.804
Completed 93800/100000 iterations, acceptance rate: 0.804
Completed 94000/100000 iterations, acceptance rate: 0.803
Completed 94200/100000 iterations, acceptance rate: 0.803
Completed 94400/100000 iterations, acceptance rate: 0.803
```

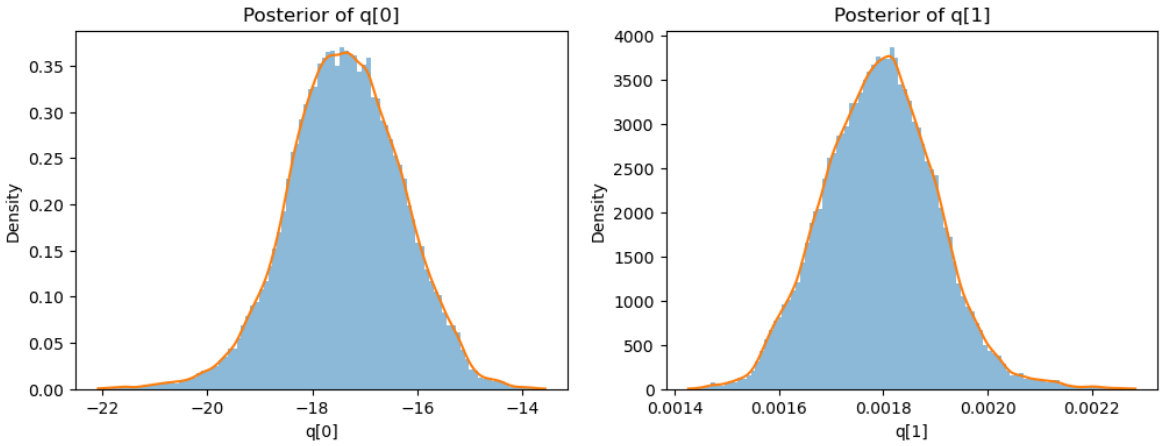
```
  Completed 94600/100000 iterations, acceptance rate: 0.803
  Completed 94800/100000 iterations, acceptance rate: 0.803
  Completed 95000/100000 iterations, acceptance rate: 0.803
  Completed 95200/100000 iterations, acceptance rate: 0.803
  Completed 95400/100000 iterations, acceptance rate: 0.803
  Completed 95600/100000 iterations, acceptance rate: 0.803
  Completed 95800/100000 iterations, acceptance rate: 0.804
  Completed 96000/100000 iterations, acceptance rate: 0.804
  Completed 96200/100000 iterations, acceptance rate: 0.804
  Completed 96400/100000 iterations, acceptance rate: 0.804
  Completed 96600/100000 iterations, acceptance rate: 0.804
  Completed 96800/100000 iterations, acceptance rate: 0.804
  Completed 97000/100000 iterations, acceptance rate: 0.804
  Completed 97200/100000 iterations, acceptance rate: 0.804
  Completed 97400/100000 iterations, acceptance rate: 0.804
  Completed 97600/100000 iterations, acceptance rate: 0.804
  Completed 97800/100000 iterations, acceptance rate: 0.804
  Completed 98000/100000 iterations, acceptance rate: 0.804
  Completed 98200/100000 iterations, acceptance rate: 0.804
  Completed 98400/100000 iterations, acceptance rate: 0.804
  Completed 98600/100000 iterations, acceptance rate: 0.804
  Completed 98800/100000 iterations, acceptance rate: 0.804
  Completed 99000/100000 iterations, acceptance rate: 0.804
  Completed 99200/100000 iterations, acceptance rate: 0.804
  Completed 99400/100000 iterations, acceptance rate: 0.804
  Completed 99600/100000 iterations, acceptance rate: 0.804
  Completed 99800/100000 iterations, acceptance rate: 0.804
  Completed 100000/100000 iterations, acceptance rate: 0.804

Final acceptance rate: 0.804

Computing posterior values for MAP estimate...

============================================================
PROBLEM 1.1 RESULTS
============================================================

MAP Estimate (q_MAP):
  Φ = -17.375551 W/cm²
  h = 0.001793 W/cm²/°C
  π(υ|q_MAP)π0(q_MAP) = 5.74e-03

Posterior Distribution π(q|υ) Summary:
============================================================
Φ:
  Mean = -17.380237 W/cm²
  Std  = 1.084073 W/cm²
  95% CI = [-19.571328, -15.320156]
```

```
h:
  Mean = 1.793447e-03 W/cm²/°C
  Std  = 1.083245e-04 W/cm²/°C
  95% CI = [1.587000e-03, 2.009767e-03]
```



In [11]:

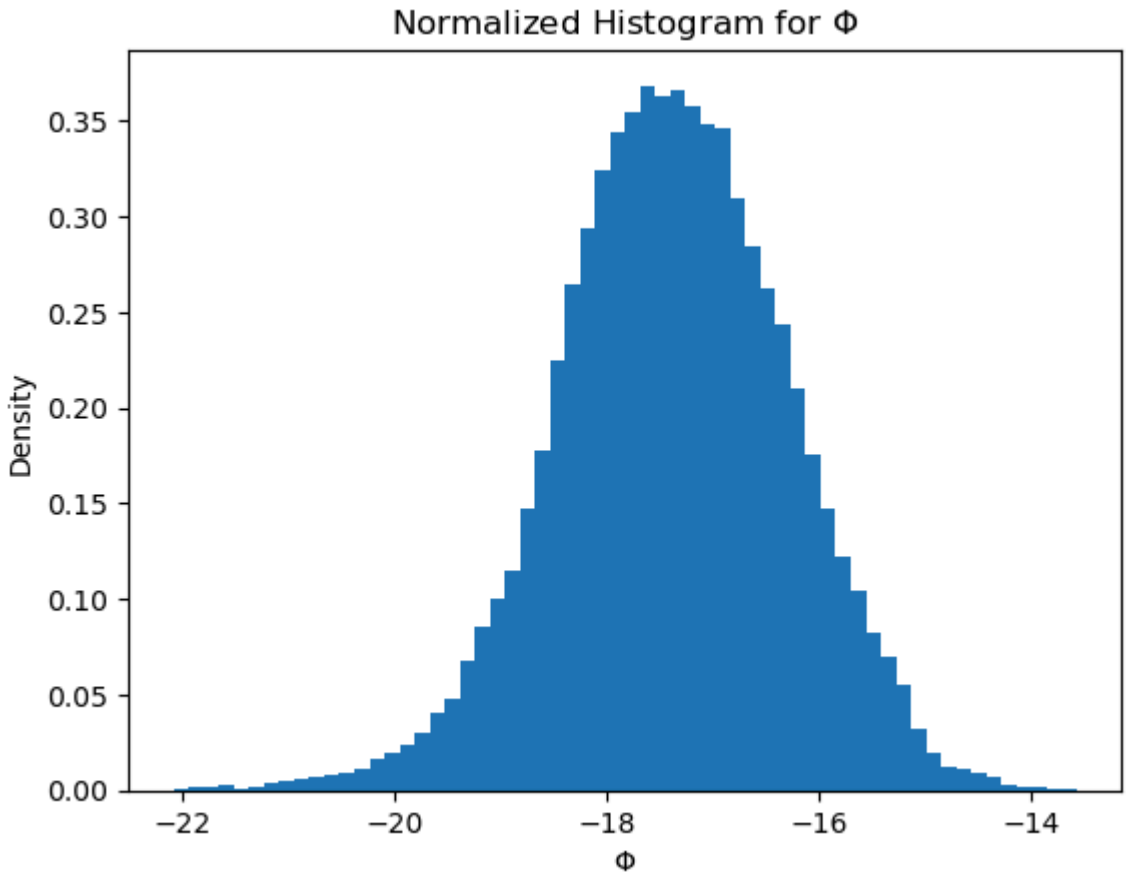
```
from scipy.stats import gaussian_kde
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for i in range(2):
    s = samples[:, i]

    # Histogram
    axes[i].hist(s, bins=100, density=True, alpha=0.5)

    # KDE smooth curve
    kde = gaussian_kde(s)
    x = np.linspace(min(s), max(s), 300)
    axes[i].plot(x, kde(x))

    axes[i].set_title(f"Posterior of q[{i}]")
    axes[i].set_xlabel(f"q[{i}]")
    axes[i].set_ylabel("Density")
```



In [19]:

```
# Normalized histograms for phi, h
plt.hist(samples[:,0], density = True, bins = 60)
plt.xlabel(r"$\Phi$")
plt.ylabel(r"Density")
plt.title(r"Normalized Histogram for $\Phi$")
```

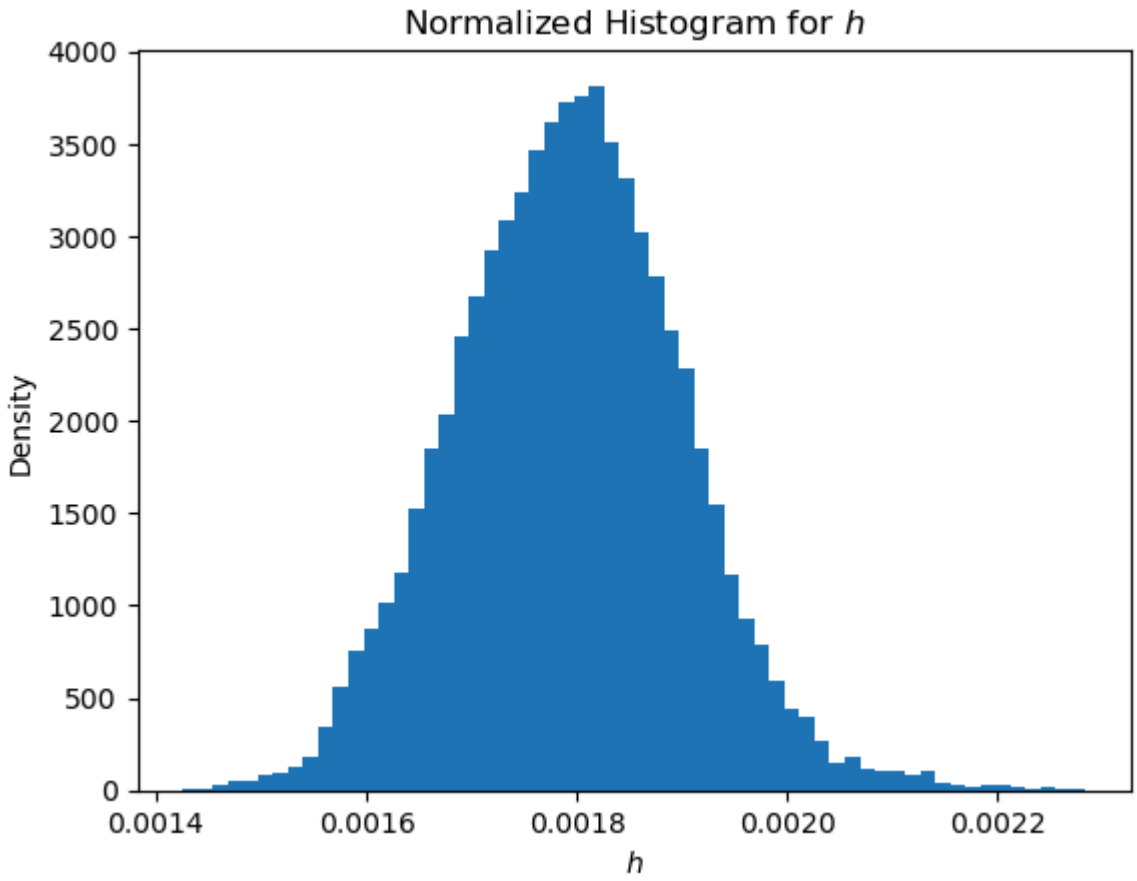
```
Text(0.5, 1.0, 'Normalized Histogram for $\\Phi$')
```



In [20]:

```
plt.hist(samples[:,1], density = True, bins = 60)
plt.xlabel(r"$h$")
plt.ylabel(r"Density")
plt.title(r"Normalized Histogram for $h$")
```

```
Text(0.5, 1.0, 'Normalized Histogram for $h$')
```



The new [22] $q_{MAP} = [\Phi = -17.375 W/cm^2, h = 0.00179 W/cm^2/°C]$

p2

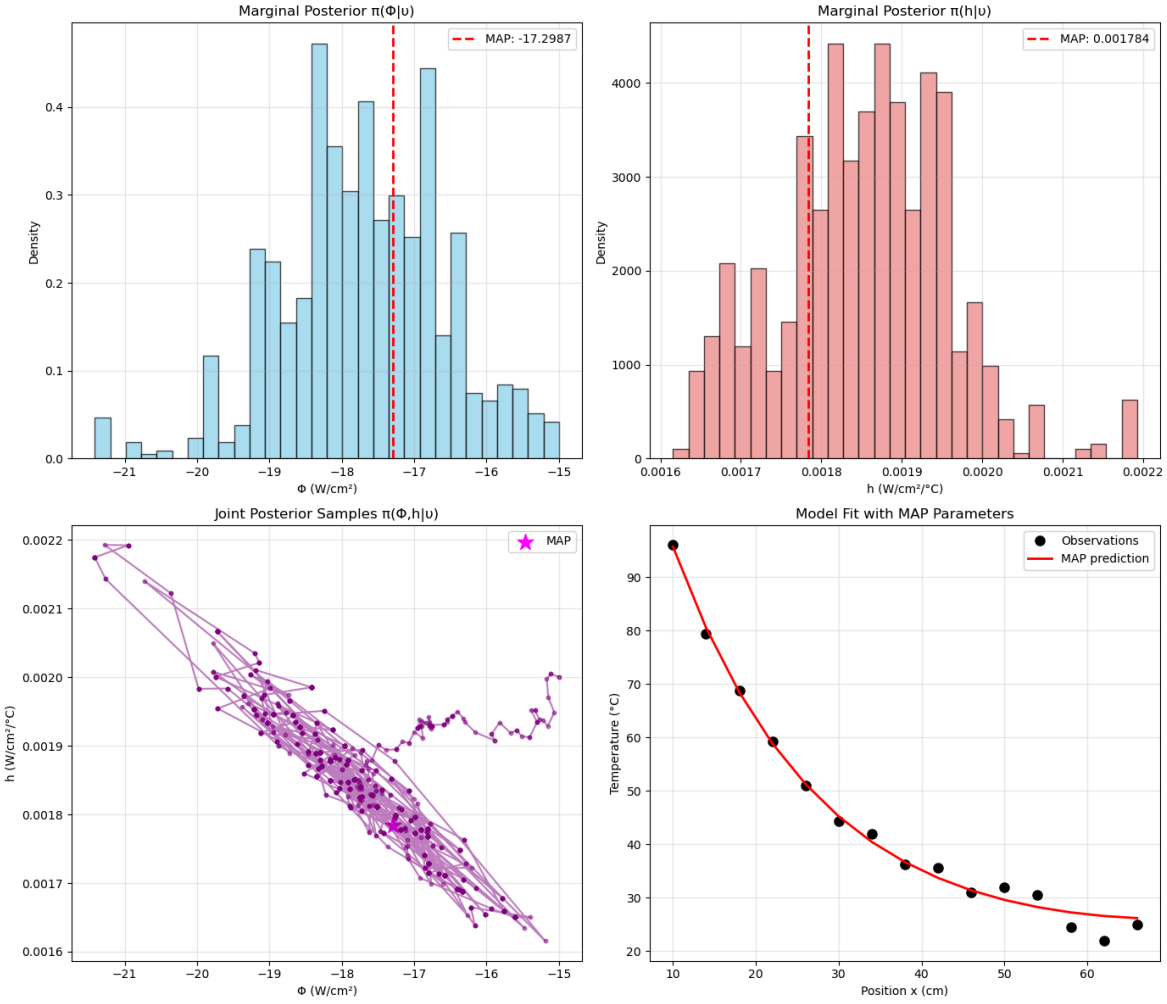
Exported 2/21/2026, 10:11:12 AM

In [5]:

```
import os, sys, numpy as np, matplotlib.pyplot as plt
sys.path.append('../hw1')
import heat_rod as hr
from mcmc import *
from am_mcmc import *
from scipy.io import loadmat

data = loadmat("HW06_Problem1.mat")
samples, posterior_values, q_map, Vk_at_101, acceptance_rate =
run_adaptive_metropolis_for_heat_equation(M=1000)

# Save results
np.savez('problem2.1_results.npz',
            samples=samples,
            q_map=q_map,
            posterior_values=posterior_values,
            acceptance_rate=acceptance_rate)

print(f"\nResults saved to 'problem2.1_results.npz'")
acceptance_rate
```

```
Shape of observations: (15,)

Running Adaptive Metropolis algorithm for 1000 iterations...
Initial proposal distribution (first 100 iterations): N(q_prev, D)
with D = diag([0.01, 2e-10])
AM parameters: sp = 2.38^2/2 = 2.8322, k0 = 100
Running Adaptive Metropolis MCMC...
  Completed 100/1000 iterations, acceptance rate: 0.677

At k=101, computed Vk from first 100 samples:
Vk = [[7.93030677e-01 3.34840269e-05]
 [3.34840269e-05 2.24989131e-09]]
  Completed 200/1000 iterations, acceptance rate: 0.432
  Completed 300/1000 iterations, acceptance rate: 0.355
  Completed 400/1000 iterations, acceptance rate: 0.296
  Completed 500/1000 iterations, acceptance rate: 0.263
  Completed 600/1000 iterations, acceptance rate: 0.259
  Completed 700/1000 iterations, acceptance rate: 0.256
  Completed 800/1000 iterations, acceptance rate: 0.253
  Completed 900/1000 iterations, acceptance rate: 0.249
  Completed 1000/1000 iterations, acceptance rate: 0.236

Final acceptance rate: 0.236

============================================================
PROBLEM 2.1 RESULTS
============================================================

At k=101, the new Vk generated from the first 100 samples is:
Vk = [[7.93030677e-01 3.34840269e-05]
 [3.34840269e-05 2.24989131e-09]]

============================================================
PROBLEM 2.2 RESULTS
============================================================

At k=M, the final Vk generated is:
Vk = [[ 1.28032133e+00 -8.54616507e-05]
 [-8.54616507e-05  1.00561129e-08]]

Computing posterior values for MAP estimate...

============================================================
PROBLEM 1.1 RESULTS
============================================================
```

```
MAP Estimate (q_MAP):
  Φ = -17.298689 W/cm²
  h = 0.001784 W/cm²/°C
  π(υ|q_MAP)π0(q_MAP) = 5.71e-03

Posterior Distribution π(q|υ) Summary:
============================================================
Φ:
  Mean = -17.715995 W/cm²
  Std  = 1.122976 W/cm²
  95% CI = [-19.779414, -15.606144]

h:
  Mean = 1.853013e-03 W/cm²/°C
  Std  = 1.024177e-04 W/cm²/°C
  95% CI = [1.654916e-03, 2.066236e-03]
```




```
Results saved to 'problem2.1_results.npz'
```

```
0.23623623623623624
```

At k=101, the new Vk generated from the first 100 samples is:

$V_k = [[7.93e^{-01}, 3.35e^{-05}]; [3.35e^{-05}, 2.25e^{-09}]]$

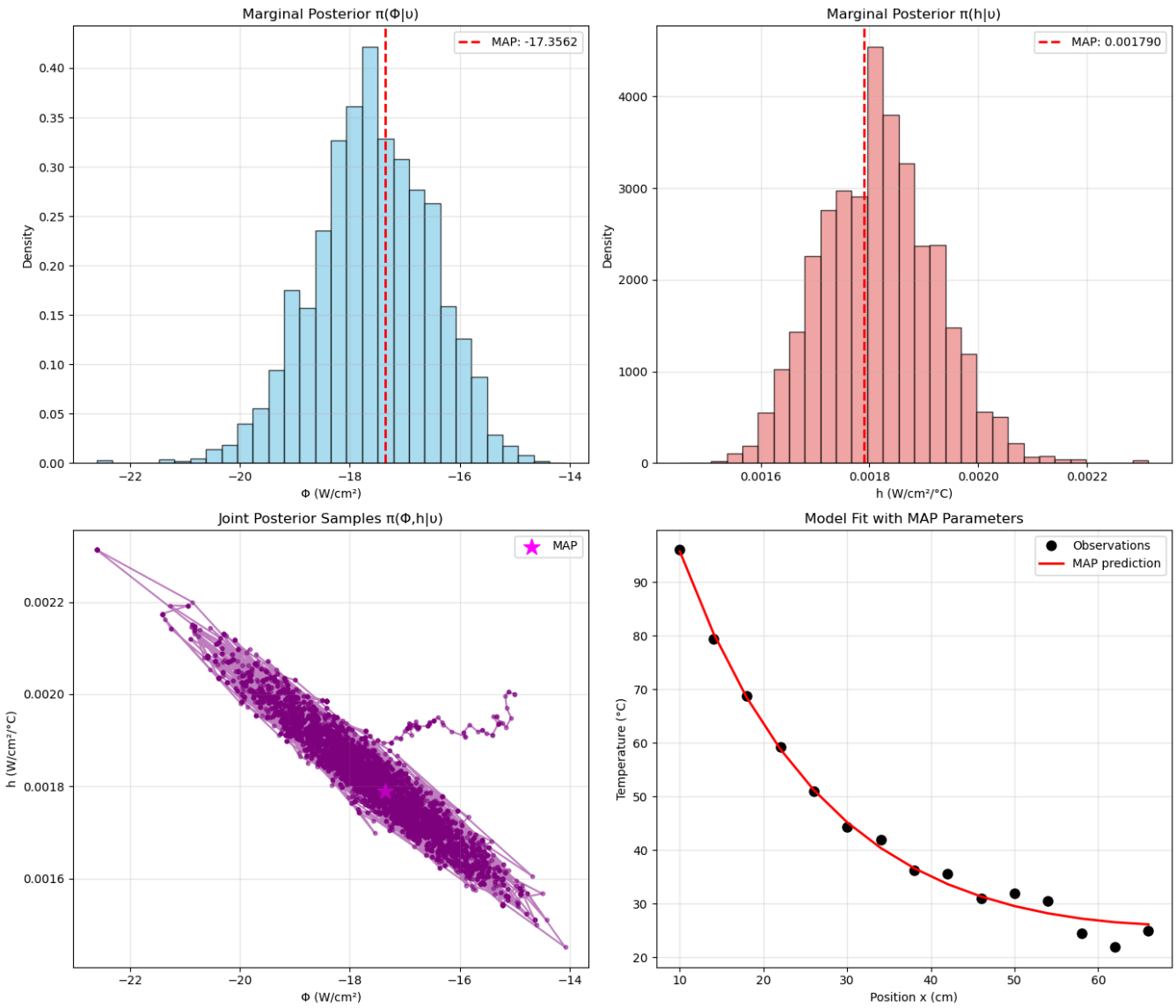
2.2

At k=M = 1000, the new Vk generated from the first 1000 samples is:

$V_k = [[1.28e^{+00}, -8.54e^{-05}]; [-8.54e^{-05}1.00e^{-08}]]$

2.3

The MAP point is [Φ = -17.298 W/cm², h = 0.00178 W/cm²/°C]. The final acceptance ratio is: 0.236.



2.4

The acceptance pattern shows a well-behaved adaptive MCMC tuning process: the acceptance rate starts high (0.43), meaning the initial proposal steps are too small, then steadily decreases as the algorithm increases the proposal scale to explore more efficiently. Around 1000–2000 iterations it stabilizes near ~~0.22–0.23, which is close to the theoretical optimal value (~~0.234 for high-dimensional random-walk Metropolis), indicating a good tuning. After that, it slowly drifts upward and stabilizes around ~0.26–0.27, suggesting adaptation continued adjusting covariance structure (not just scale), slightly improving move efficiency. Overall, this shows convergence from an over-conservative proposal to a near-optimal one, followed by mild fine-tuning and stabilization.

With 100,000 samples from the previous problem, the most accurate estimate of q_{MAP} is [22] $[\Phi = -17.375W/cm^2, h = 0.00179W/cm^2/°C]$. From the previous MCMC algorithm, with 1000 samples, we get [22] $q_{MAP,MCMC,1000} = [\Phi = -17.560W/cm^2, h = 0.0018W/cm^2/°C]$. Now, with the adaptive algorithm and a sample size of 1000, we get The MAP point is [22] $q_{MAP,AM-MCMC,1000} = [\Phi = -17.298W/cm^2, h = 0.0018W/cm^2/°C]$. Notice that with the same number of samples, the adaptive method gives a far better MAP estimate than the standard MCMC method.

Using Adaptive Metropolis (AM) significantly improves efficiency compared to standard Metropolis--Hastings with a fixed proposal covariance D. In standard MH, performance is highly sensitive to the manual choice of D; if D is poorly scaled, the chain either moves too slowly (high acceptance but strong correlation) or rejects too often (low acceptance and poor exploration). With AM, the proposal covariance V_k adapts to the empirical covariance of the samples, automatically learning the posterior scale and correlation structure between Φ and h. Consequently, the acceptance ratio typically stabilizes near the optimal range (approximately 0.23--0.30 for $p = 2$), mixing improves, autocorrelation decreases, and the chain reaches the high-density region more quickly (shorter effective burn-in). The MAP estimate itself generally does not change very significantly, since the target distribution is unchanged, but it is obtained more reliably and efficiently with improved exploration of the posterior. Overall, AM increases robustness, reduces manual tuning, and yields more efficient sampling than fixed-covariance Metropolis--Hastings.

p3_1

Exported 2/21/2026, 10:11:20 AM

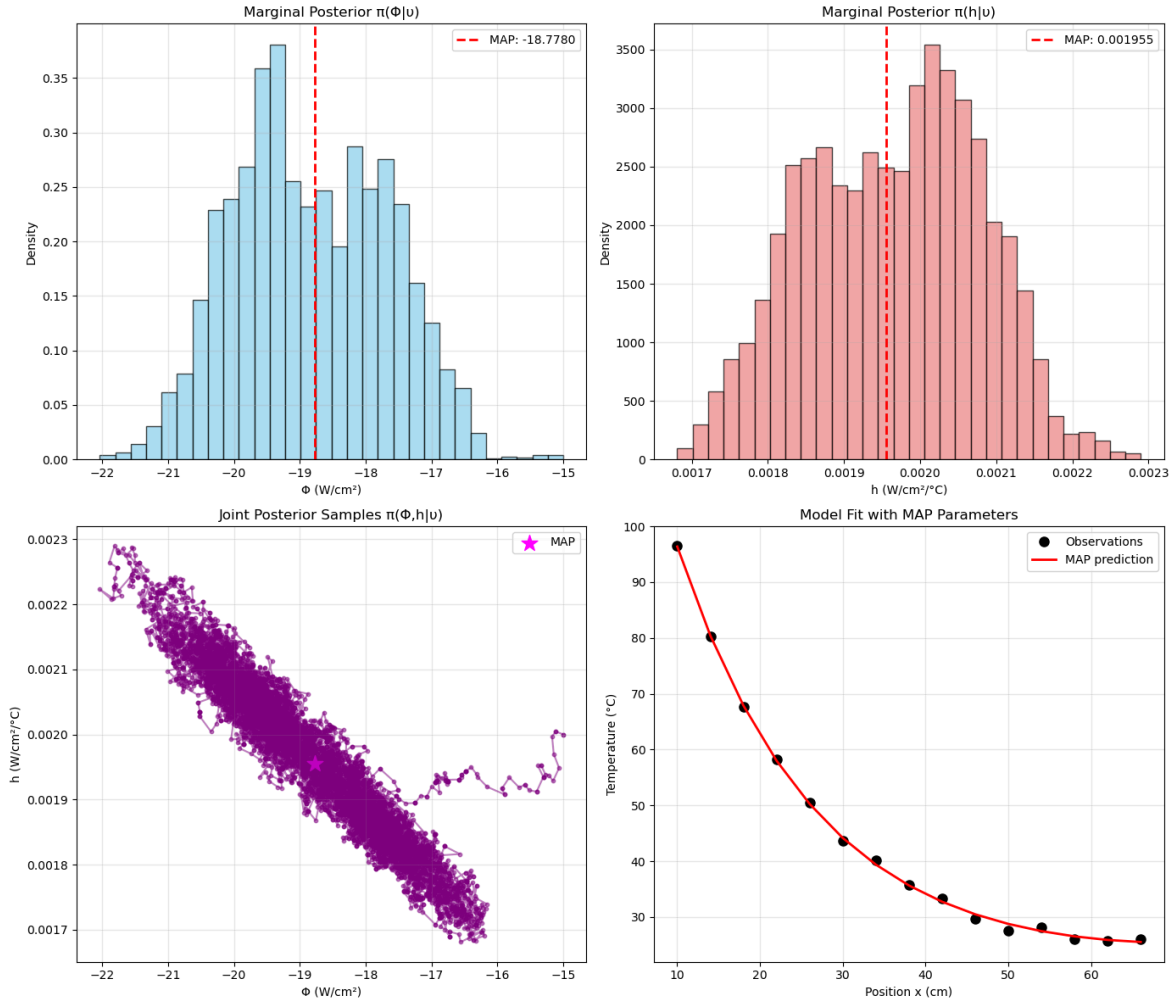
3.1

In [2]:

```
import os, sys, numpy as np, matplotlib.pyplot as plt
sys.path.append('../hw1')
import heat_rod as hr
from mcmc import *
from scipy.io import loadmat

data_name ="HW06_Problem3.mat"
samples, q_map, posterior_values, acceptance_rate =
run_metropolis_for_heat_equation(data_name = data_name, M = 10000)

# Save results
np.savez('problem3.1_results.npz',
            samples=samples,
            q_map=q_map,
            posterior_values=posterior_values,
            acceptance_rate=acceptance_rate)

print(f"\nResults saved to 'problem3.1_results.npz'")
```

```
Shape of observations: (15,)
Initial test - r(q0, q0) = 1.000000

Running Metropolis algorithm for 10000 iterations...
Proposal distribution: N(q_prev, D) with D = diag([0.01, 2e-10])
Running Metropolis-Hastings MCMC...
  Completed 200/10000 iterations, acceptance rate: 0.729
  Completed 400/10000 iterations, acceptance rate: 0.772
  Completed 600/10000 iterations, acceptance rate: 0.798
  Completed 800/10000 iterations, acceptance rate: 0.814
  Completed 1000/10000 iterations, acceptance rate: 0.818
  Completed 1200/10000 iterations, acceptance rate: 0.826
  Completed 1400/10000 iterations, acceptance rate: 0.832
  Completed 1600/10000 iterations, acceptance rate: 0.836
  Completed 1800/10000 iterations, acceptance rate: 0.840
  Completed 2000/10000 iterations, acceptance rate: 0.835
  Completed 2200/10000 iterations, acceptance rate: 0.834
  Completed 2400/10000 iterations, acceptance rate: 0.835
  Completed 2600/10000 iterations, acceptance rate: 0.838
  Completed 2800/10000 iterations, acceptance rate: 0.835
  Completed 3000/10000 iterations, acceptance rate: 0.835
  Completed 3200/10000 iterations, acceptance rate: 0.834
  Completed 3400/10000 iterations, acceptance rate: 0.833
  Completed 3600/10000 iterations, acceptance rate: 0.832
  Completed 3800/10000 iterations, acceptance rate: 0.828
  Completed 4000/10000 iterations, acceptance rate: 0.827
  Completed 4200/10000 iterations, acceptance rate: 0.826
  Completed 4400/10000 iterations, acceptance rate: 0.826
  Completed 4600/10000 iterations, acceptance rate: 0.826
  Completed 4800/10000 iterations, acceptance rate: 0.825
  Completed 5000/10000 iterations, acceptance rate: 0.826
  Completed 5200/10000 iterations, acceptance rate: 0.827
  Completed 5400/10000 iterations, acceptance rate: 0.827
  Completed 5600/10000 iterations, acceptance rate: 0.826
  Completed 5800/10000 iterations, acceptance rate: 0.826
  Completed 6000/10000 iterations, acceptance rate: 0.826
  Completed 6200/10000 iterations, acceptance rate: 0.824
  Completed 6400/10000 iterations, acceptance rate: 0.825
  Completed 6600/10000 iterations, acceptance rate: 0.826
  Completed 6800/10000 iterations, acceptance rate: 0.826
  Completed 7000/10000 iterations, acceptance rate: 0.826
  Completed 7200/10000 iterations, acceptance rate: 0.825
  Completed 7400/10000 iterations, acceptance rate: 0.825
  Completed 7600/10000 iterations, acceptance rate: 0.825
  Completed 7800/10000 iterations, acceptance rate: 0.824
  Completed 8000/10000 iterations, acceptance rate: 0.823
```

```
  Completed 8200/10000 iterations, acceptance rate: 0.822
  Completed 8400/10000 iterations, acceptance rate: 0.821
  Completed 8600/10000 iterations, acceptance rate: 0.821
  Completed 8800/10000 iterations, acceptance rate: 0.821
  Completed 9000/10000 iterations, acceptance rate: 0.822
  Completed 9200/10000 iterations, acceptance rate: 0.822
  Completed 9400/10000 iterations, acceptance rate: 0.821
  Completed 9600/10000 iterations, acceptance rate: 0.821
  Completed 9800/10000 iterations, acceptance rate: 0.820
  Completed 10000/10000 iterations, acceptance rate: 0.820

Final acceptance rate: 0.820

Computing posterior values for MAP estimate...

============================================================
PROBLEM 1.1 RESULTS
============================================================

MAP Estimate (q_MAP):
  Φ = -18.778007 W/cm²
  h = 0.001955 W/cm²/°C
  π(υ|q_MAP)π0(q_MAP) = 1.61e+00

Posterior Distribution π(q|υ) Summary:
------------------------------------------------------------
Φ:
  Mean = -18.830713 W/cm²
  Std  = 1.159654 W/cm²
  95% CI = [-20.887713, -16.661970]

h:
  Mean = 1.964490e-03 W/cm²/°C
  Std  = 1.145543e-04 W/cm²/°C
  95% CI = [1.748568e-03, 2.164453e-03]
```




```
Results saved to 'problem3.1_results.npz'
```

$^{22}q_{MAP} = [-18.72W/cm^2; h = 0.00195W/cm^2/°C]$ after removing 300 burn-in samples.

In [3]:

```
burnin = 300
q_map_after_burnin = samples[np.argmax(posterior_values[burnin:])]
print(f"MAP point after removing {burnin} burn-in points",
q_map_after_burnin)
print(f"MAP point wihout removing {burnin} burn-in points", q_map)
```

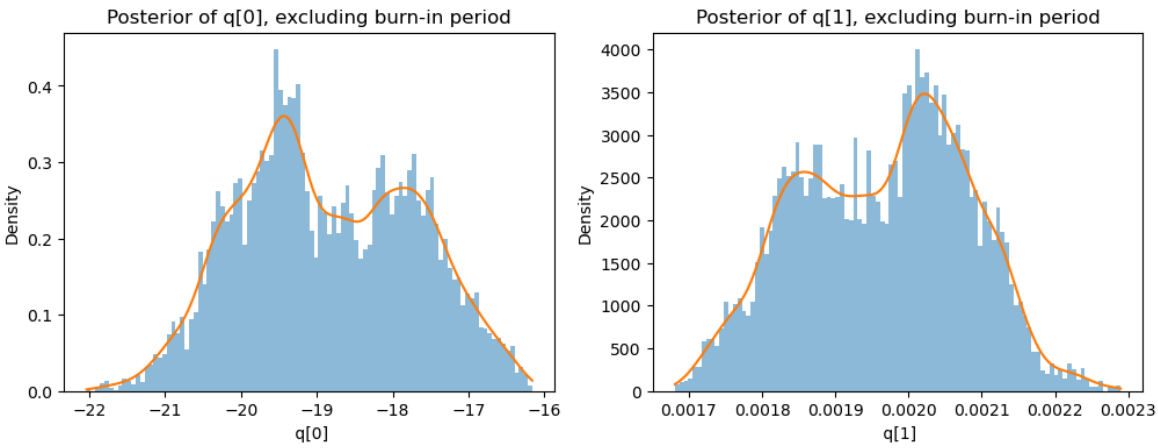
```
MAP point after removing 300 burn-in points [-1.87244917e+01
1.95073235e-03]
MAP point wihout removing 300 burn-in points [-1.87780074e+01
1.95508105e-03]
```

Histograms of Samples

KDE of PDF

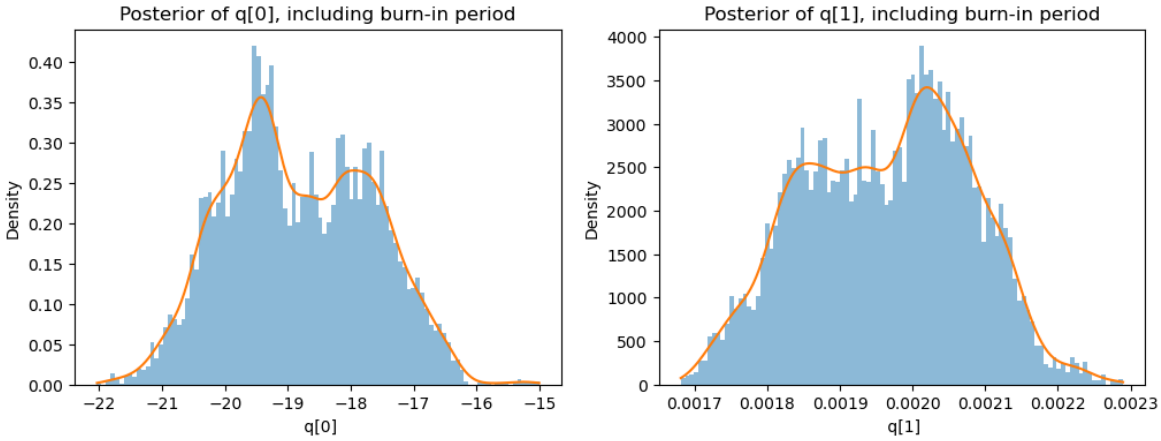
In [5]:

```
from scipy.stats import gaussian_kde
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for i in range(2):
    s = samples[burnin:, i] ####TODO: BURNIN INCLUDED?

    # Histogram
    axes[i].hist(s, bins=100, density=True, alpha=0.5)

    # KDE smooth curve
    kde = gaussian_kde(s)
    x = np.linspace(min(s), max(s), 300)
    axes[i].plot(x, kde(x))

    axes[i].set_title(f"Posterior of q[{i}], excluding burn-in
period")
    axes[i].set_xlabel(f" q[{i}]")
    axes[i].set_ylabel("Density")
```



In [6]:

```
from scipy.stats import gaussian_kde
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for i in range(2):
    s = samples[:, i] ####TODO: BURNIN INCLUDED?

    # Histogram
    axes[i].hist(s, bins=100, density=True, alpha=0.5)

    # KDE smooth curve
    kde = gaussian_kde(s)
    x = np.linspace(min(s), max(s), 300)
    axes[i].plot(x, kde(x))

    axes[i].set_title(f"Posterior of q[{i}], including burn-in
period")
    axes[i].set_xlabel(f" q[{i}]")
    axes[i].set_ylabel("Density")
```



Note: q[0] is ϕ, while q[1] is h.

p3_2

Exported 2/21/2026, 10:26:46 AM

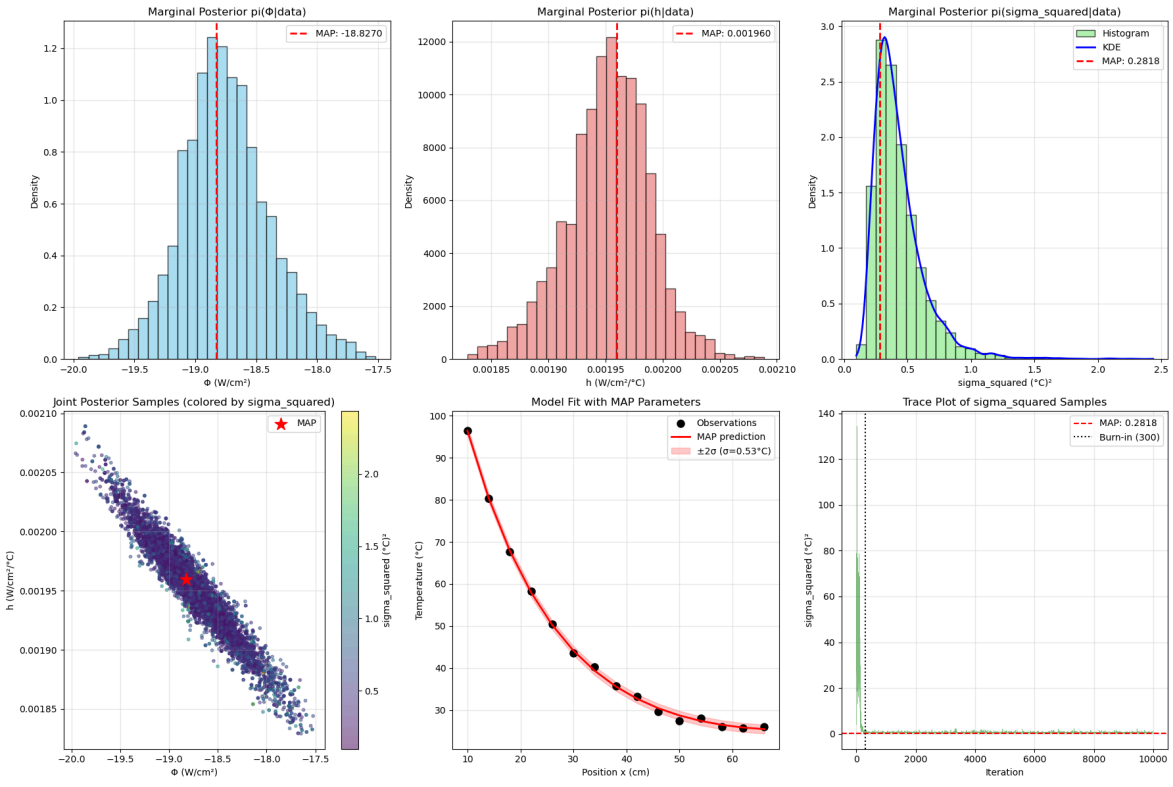
3.2

In [5]:

```
import os, sys, numpy as np, matplotlib.pyplot as plt
sys.path.append('../hw1')
import heat_rod as hr
from mcmc import *
from gibbs_mcmc import *
from scipy.io import loadmat

burnin = 300
samples_gibbs, s2_samples_gibbs, q_map_gibbs, s2_map_gibbs,
posterior_values_gibbs, acc_rate_gibbs =
run_metropolis_gibbs_for_heat_equation(M=10000, burnin=burnin)
```

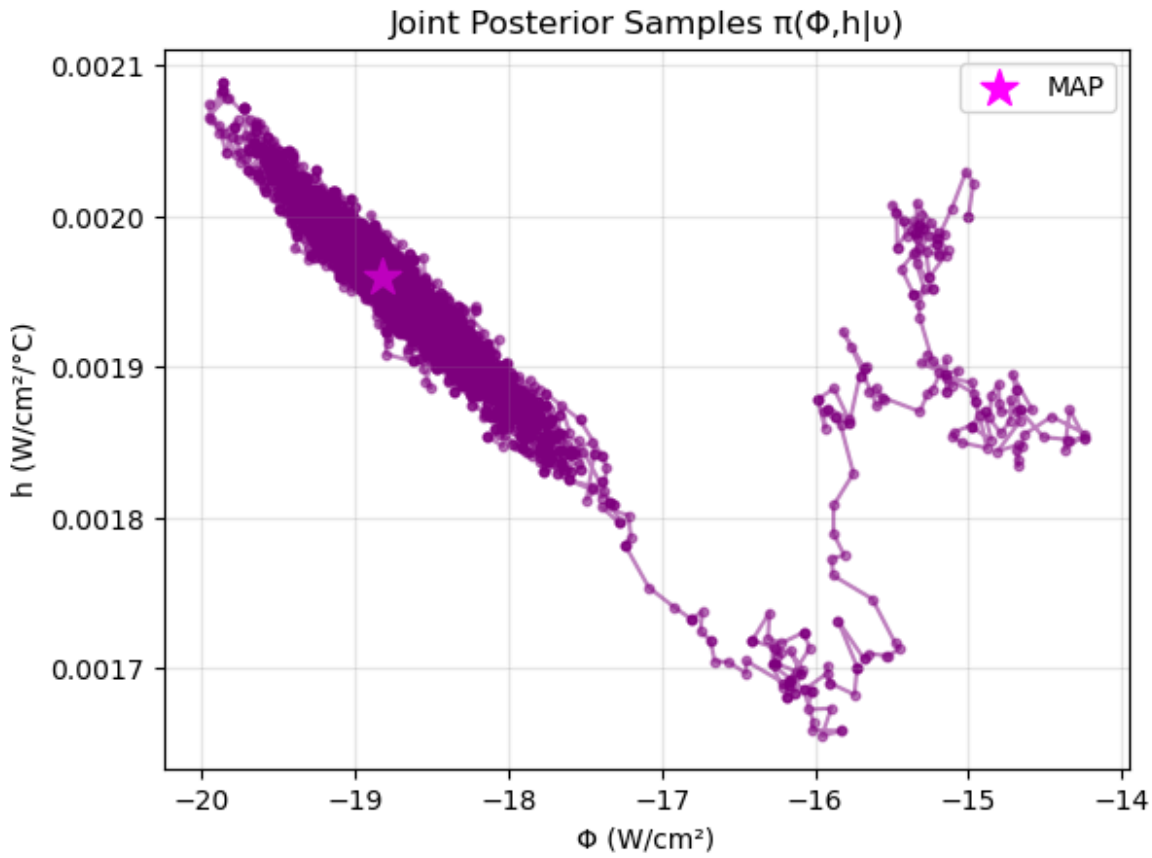
```
Shape of observations: (15,)

======================================================================
PROBLEM 1.2: METROPOLIS-HASTINGS WITH GIBBS FOR UNKNOWN sigma_squared
======================================================================
Hyperparameters: ns = 0.01, sigma_squared_s = 4.0
Initial sigma_squared = 4.0
Running for M = 10000 iterations with 300 burn-in samples
Running Metropolis-Hastings with Gibbs sampling for unknown
sigma_squared...
ns = 0.01, sigma2_s = 4.0
Initial sigma_squared = 4.0
  Completed 1000/10000 iterations, MCMC acceptance rate: 0.586,
sigma_squared = 0.2674
  Completed 2000/10000 iterations, MCMC acceptance rate: 0.541,
sigma_squared = 0.4642
  Completed 3000/10000 iterations, MCMC acceptance rate: 0.538,
sigma_squared = 0.6575
  Completed 4000/10000 iterations, MCMC acceptance rate: 0.537,
sigma_squared = 0.4546
  Completed 5000/10000 iterations, MCMC acceptance rate: 0.539,
sigma_squared = 0.7179
  Completed 6000/10000 iterations, MCMC acceptance rate: 0.536,
sigma_squared = 0.2431
  Completed 7000/10000 iterations, MCMC acceptance rate: 0.536,
sigma_squared = 0.5990
  Completed 8000/10000 iterations, MCMC acceptance rate: 0.532,
sigma_squared = 0.5049
  Completed 9000/10000 iterations, MCMC acceptance rate: 0.529,
sigma_squared = 0.2960
  Completed 10000/10000 iterations, MCMC acceptance rate: 0.528,
sigma_squared = 0.2413

Final acceptance rate for q: 0.528

Computing posterior values for MAP estimate...

============================================================
PROBLEM 1.2 RESULTS (with unknown sigma_squared)
============================================================

MAP Estimate (q_MAP, sigma_squared_MAP):
  Φ = -18.826955 W/cm²
  h = 0.001960 W/cm²/°C
  sigma_squared = 0.281789 (°C)²
  sigma = 0.5308 °C
```

```
  pi(data|q_MAP,sigma_squared_MAP)pi0(q_MAP,sigma_squared_MAP) =
3.12e-05

Posterior Distribution pi(q,sigma_squared|data) Summary (after burn-
in):
============================================================
Φ:
  Mean = -18.748429 W/cm²
  Std  = 0.363946 W/cm²
  95% CI = [-19.448307, -17.954186]

h:
  Mean = 1.952216e-03 W/cm²/°C
  Std  = 3.695653e-05 W/cm²/°C
  95% CI = [1.871018e-03, 2.024902e-03]

sigma_squared:
  Mean = 0.431687 (°C)²
  Std  = 0.205363 (°C)²
  95% CI = [0.192345, 0.951567]
```



In [6]:

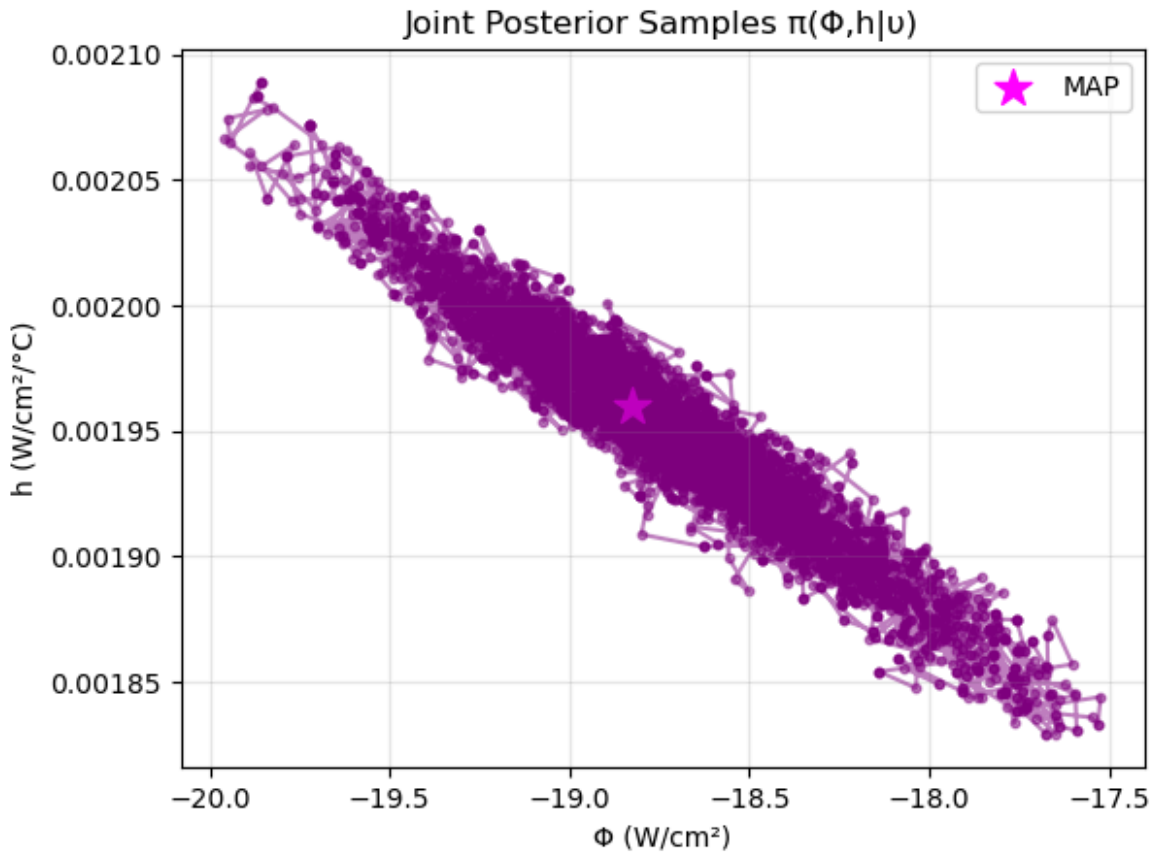
```
plt.scatter(samples_gibbs[:, 0], samples_gibbs[:, 1], alpha=0.5,
c='purple', s=10)
plt.plot(samples_gibbs[:, 0], samples_gibbs[:, 1], alpha=0.5,
c='purple')

plt.scatter(q_map_gibbs[0], q_map_gibbs[1], color='magenta', s=200,
marker='*', label='MAP')

plt.xlabel('Φ (W/cm²)')
plt.ylabel('h (W/cm²/°C)')
plt.title('Joint Posterior Samples π(Φ,h|υ)')
plt.legend()
plt.grid(True, alpha=0.3)
```



removing burn-in

In [8]:

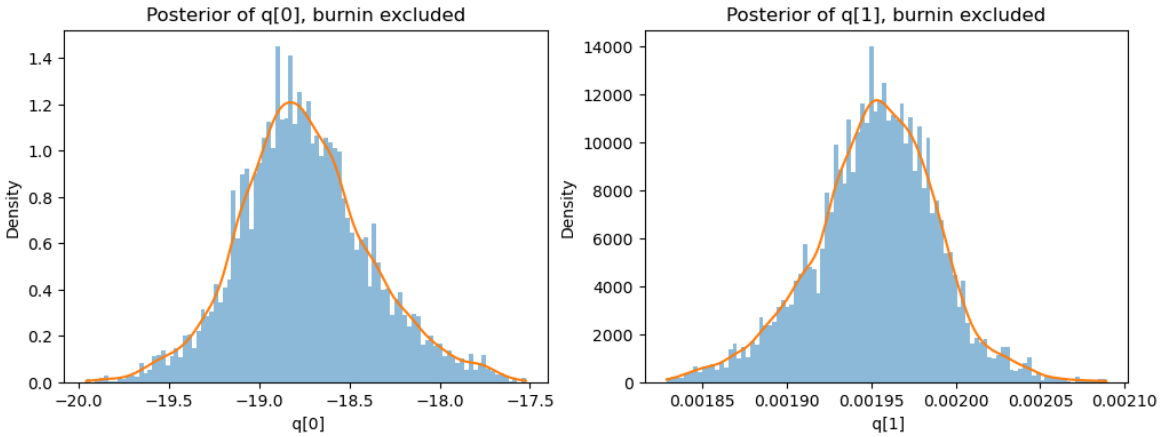
```
plt.scatter(samples_gibbs[burnin:, 0], samples_gibbs[burnin:, 1],
alpha=0.5, c='purple', s=10)
plt.plot(samples_gibbs[burnin:, 0], samples_gibbs[burnin:, 1],
alpha=0.5, c='purple')

plt.scatter(q_map_gibbs[0], q_map_gibbs[1], color='magenta', s=200,
marker='*', label='MAP')

plt.xlabel('Φ (W/cm²)')
plt.ylabel('h (W/cm²/°C)')
plt.title('Joint Posterior Samples π(Φ,h|υ)')
plt.legend()
plt.grid(True, alpha=0.3)
```



$^{22}q_{MAP,\,Gibbs} = [-18.83W\,/\,cm^2; h = 0.00196W\,/\,cm^2\,/\,°C]$ with $^{2}\sigma^2 = 0.282(°C)^2$ after removing 300 burn-in samples.

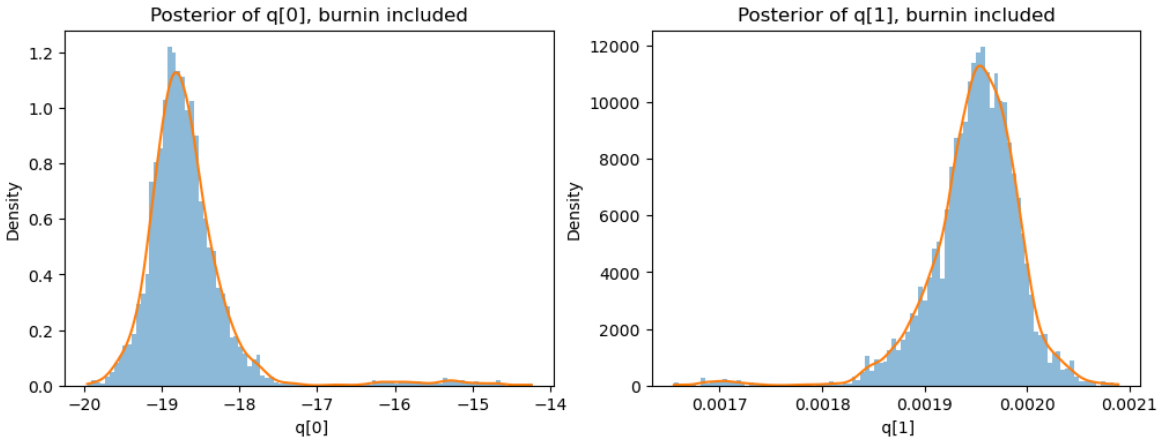
In [10]:

```
from scipy.stats import gaussian_kde
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for i in range(2):
    s = samples_gibbs[burnin:, i] ####TODO: BURNIN INCLUDED?

    # Histogram
    axes[i].hist(s, bins=100, density=True, alpha=0.5)

    # KDE smooth curve
    kde = gaussian_kde(s)
    x = np.linspace(min(s), max(s), 300)
    axes[i].plot(x, kde(x))

    axes[i].set_title(f"Posterior of q[{i}], burnin excluded")
    axes[i].set_xlabel(f" q[{i}]")
    axes[i].set_ylabel("Density")
```



In [11]:

```
from scipy.stats import gaussian_kde
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for i in range(2):
    s = samples_gibbs[:, i] ####TODO: BURNIN INCLUDED?

    # Histogram
    axes[i].hist(s, bins=100, density=True, alpha=0.5)

    # KDE smooth curve
    kde = gaussian_kde(s)
    x = np.linspace(min(s), max(s), 300)
    axes[i].plot(x, kde(x))

    axes[i].set_title(f"Posterior of q[{i}], burnin included")
    axes[i].set_xlabel(f" q[{i}]")
    axes[i].set_ylabel("Density")
```



Note: q[0] is ϕ, while q[1] is h.

The exclusion of burn-in samples leads to more symmetric distributions because the early iterations of the MCMC chain are heavily influenced by the initial starting point and have not yet converged to the true posterior distribution. During burn-in, the chain is still "warming up" and exploring regions far from the high-probability area, often exhibiting transient behavior, high autocorrelation, and directional drift as it moves from the initial guess toward the stationary distribution. These early samples tend to be biased toward one side of the posterior (depending on the starting location) and can skew the histogram, creating asymmetry and heavier tails. Once these non-representative samples are removed, the remaining samples are drawn from a more stationary distribution where the chain has reached equilibrium and is properly sampling around the true posterior mode. This equilibrium sampling produces a more symmetric distribution that better reflects the actual uncertainty in the parameters, as the chain has now explored the parameter space in a balanced manner without the directional bias inherent in the transient phase.

3.3

The posterior distributions for both Φ and h exhibit smoother characteristics with reduced variance when σ^2 is estimated rather than fixed. This smoothing effect occurs because the Gibbs sampler allows the model to adaptively adjust the error variance based on the current parameter estimates, effectively weighting observations according to their consistency with the model. Mathematically, this adaptivity arises from the conditional structure:

$$p(q \mid v, \sigma^2) \propto \exp\Big(-\frac{1}{2\sigma^2} \sum_{i=1}^{n} (v_i - f_i(q))^2 \Big)\, \pi_0(q)$$

$$p(\sigma^2 \mid v, q) = Inv\text{–}Gamma\Big(\frac{n_s + n}{2}, \frac{n_s \sigma_s^2 + SS_q}{2}\Big)$$

The burn-in period becomes more critical in the Gibbs implementation, typically requiring more iterations to achieve stationarity as the chain must now explore the joint space of both parameters and the error variance simultaneously. The Markov chain in the Φ-h plane shows increased exploration during early iterations as the uncertainty in σ^2 propagates through the acceptance probability calculations:

$$r(q^*, q_{k-1}) = \frac{\pi(v \mid q^*, \sigma^2_{k-1})\, \pi_0(q^*)}{\pi(v \mid q_{k-1}, \sigma^2_{k-1})\, \pi_0(q_{k-1})}$$

The most significant improvement appears in the model's ability to quantify uncertainty—the credible intervals for predictions properly expand when the model fits poorly and contract when the fit is good, whereas the fixed σ^2 approach maintains constant uncertainty bands regardless of local fit quality.

The Gibbs step adds minimal computational overhead per iteration—only requiring calculation of $SS_{\mathbf{q}}$ and one gamma random variate. However, the improved mixing and more realistic uncertainty quantification often justify this small additional cost. The marginal posterior of σ^2 itself provides valuable diagnostic information: if the true error variance differs substantially from the assumed fixed value, the σ^2 samples will concentrate away from that value, revealing model misspecification. Additionally the Gibbs samples are always accepted (no computational resource waste due to rejected samples).

In practice, estimating σ^2 leads to more robust inference when the true error variance is unknown, which is generally the case in real applications. The fixed σ^2 approach risks either over-confident predictions (if the assumed variance is too small) or unnecessarily conservative estimates (if assumed too large). The Gibbs sampling approach automatically calibrates this trade-off based on the data.

```python
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat
from scipy.stats import norm, multivariate_normal
sys.path.append('../hw1')
import heat_rod as hr

def MetropolisHastingsMCMC(q0, J_func, r_calc, M, rng_seed=None):
    """
    General Metropolis-Hastings MCMC function

    Parameters:
    q0: initial sample
    J_func: proposal distribution function J(q* | q_{k-1})
    r_calc: function to compute acceptance ratio r(q*, q_{k-1})
    M: number of samples to generate
    rng_seed: random seed for reproducibility

    Returns:
    samples: array of MCMC samples
    acceptance_rate: acceptance rate of the chain
    """
    if rng_seed is not None:
        np.random.seed(rng_seed)

    q_current = np.array(q0, dtype=float)
    samples = np.zeros((M, len(q0)))
    samples[0] = q_current

    accepted = 0

    print("Running Metropolis-Hastings MCMC...")
    for i in range(1, M):
        # Step 2.1: Generate proposal from J(q* | q_{k-1})
        q_proposed = J_func(q_current)

        # Step 2.2: Compute the ratio r(q*, q_{k-1})
        r = r_calc(q_proposed, q_current)

        # Step 2.3: Accept with probability min(1, r)
        if np.random.random() < min(1, r):
            q_current = q_proposed
            accepted += 1

        samples[i] = q_current

        if (i + 1) % 200 == 0:
            print(f"  Completed {i + 1}/{M} iterations, acceptance rate: {accepted/(i):.3f}")

    acceptance_rate = accepted / (M - 1)

    return samples, acceptance_rate

def setup_heat_equation_problem(mat_file='HW06_Problem1.mat'):
    """
    Set up the heat equation problem using data from MATLAB file
    """
    data = loadmat(mat_file)

    obs_data = data['observations'].flatten()
    print(f"Shape of observations: {obs_data.shape}")
    T_obs = obs_data

    # Observation locations (as given in the problem)
    x0 = 10   # cm
    dx = 4    # spacing
    x_obs = x0 + np.arange(len(obs_data)) * dx

    # Rod parameters
    a = 0.95  # cm
    b = 0.95  # cm
    k = 2.3   # W/cm/C
    Tamb = 21.29  # C
    L = 70    # cm

    q0 = np.array([-15.0, 0.002])  # [Phi, h]
    phi0, h0 = q0

    rod = hr.SteadyStateRod(a, b, phi0, k, h0, Tamb, L)

    # Known error variance
    sigma2_0 = 4.0  # (2 C)^2

    return rod, x_obs, T_obs, sigma2_0, q0

def create_posterior_ratio_func(rod, x_obs, T_obs, sigma2_0):
    """
    Create function to compute r(q*, q_{k-1}) = П‰(q*|Oμ) / П‰(q_{k-1}|Oμ)
    For Metropolis algorithm with symmetric proposal, this is the acceptance ratio
    """
    # Prior parameters
    prior_Phi_mean = -15
    prior_Phi_std = 10
    prior_h_mean = 0.001
    prior_h_std = 0.005

    def posterior_ratio(q_star, q_prev):
        """
        Compute r(q*, q_{k-1}) = [ПЂ(П…|q*)ПЂ0(q*)] / [ПЂ(П…|q_{k-1})ПЂ0(q_{k-1})]
        """
        # Evaluate at q_star
        Phi_star, h_star = q_star
        rod.Phi_val = Phi_star
        rod.h_val = h_star
        T_model_star = rod.Ts(x_obs)

        # Log likelihood for q_star (using log to avoid underflow, then exponentiate)
        n = len(x_obs)
        log_likelihood_star = -0.5 * np.sum((T_obs - T_model_star)**2 / sigma2_0)

        # Log prior for q_star
        log_prior_star = (norm.logpdf(Phi_star, prior_Phi_mean, prior_Phi_std) +
                          norm.logpdf(h_star, prior_h_mean, prior_h_std))

        # Evaluate at q_prev
        Phi_prev, h_prev = q_prev
        rod.Phi_val = Phi_prev
        rod.h_val = h_prev
        T_model_prev = rod.Ts(x_obs)

        # Log likelihood for q_prev
```

```
        log_likelihood_prev = -0.5 * np.sum((T_obs - T_model_prev)**2 / sigma2_0)

        # Log prior for q_prev
        log_prior_prev = (norm.logpdf(Phi_prev, prior_Phi_mean, prior_Phi_std) +
                          norm.logpdf(h_prev, prior_h_mean, prior_h_std))

        # Log ratio
        log_ratio = (log_likelihood_star + log_prior_star) - (log_likelihood_prev + log_prior_prev)

        # Return actual ratio (exponentiate)
        return np.exp(log_ratio)

    return posterior_ratio

def run_metropolis_for_heat_equation(D =None, M = None, data_name = None):
    """
    Run Metropolis algorithm for the heat equation problem (Problem 1.1)
    """
    if data_name is None:
        data_name = 'HW06_Problem1.mat'

    # Setup problem
    rod, x_obs, T_obs, sigma2_0, q0 = setup_heat_equation_problem(data_name)
    if D is None:

        D = np.array([[1e-2, 0],
                      [0, 2e-10]])

    # Create proposal function J(q* | q_{k-1}) = N(q_{k-1}, D)
    def proposal_func(q_current):
        return np.random.multivariate_normal(q_current, D)

    # Create function to compute r(q*, q_{k-1})
    posterior_ratio_func = create_posterior_ratio_func(rod, x_obs, T_obs, sigma2_0)

    # Test at initial point
    initial_ratio = posterior_ratio_func(q0, q0)  # Should be 1.0
    print(f"Initial test - r(q0, q0) = {initial_ratio:.6f}")

    if M is None:
        M = 1000
    print(f"\nRunning Metropolis algorithm for {M} iterations...")
    print(f"Proposal distribution: N(q_prev, D) with D = diag([{D[0,0]}, {D[1,1]}])")

    samples, acceptance_rate = MetropolisHastingsMCMC(
        q0,
        proposal_func,
        posterior_ratio_func,
        M,
        rng_seed=42
    )

    print(f"\nFinal acceptance rate: {acceptance_rate:.3f}")

    # Find MAP estimate
    # We need to recompute the posterior (likelihood * prior) for each sample
    print("\nComputing posterior values for MAP estimate...")

    # Prior parameters
    prior_Phi_mean = -15
    prior_Phi_std = 10
    prior_h_mean = 0.001
    prior_h_std = 0.005

    # Compute unnormalized posterior for each sample
    posterior_values = np.zeros(M)
    for i, q in enumerate(samples):
        Phi, h = q
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)

        # Likelihood
        n = len(x_obs)
        likelihood = np.exp(-0.5 * np.sum((T_obs - T_model)**2 / sigma2_0))

        # Prior
        prior = (norm.pdf(Phi, prior_Phi_mean, prior_Phi_std) *
                 norm.pdf(h, prior_h_mean, prior_h_std))

        # Unnormalized posterior (pi(data|q)pi0(q))
        posterior_values[i] = likelihood * prior

    # Find MAP estimate (maximum unnormalized posterior)
    map_idx = np.argmax(posterior_values)
    q_map = samples[map_idx]
    post_map = posterior_values[map_idx]

    print(f"\n{'='*60}")
    print("PROBLEM 1.1 RESULTS")
    print(f"{'='*60}")
    print(f"\nMAP Estimate (q_MAP):")
    print(f"  O¦ = {q_map[0]:.6f} W/cmBI")
    print(f"  h = {q_map[1]:.6f} W/cmBI/B°C")
    print(f"  ΠЪ(П…|q_MAP)ΠЪ0(q_MAP) = {post_map:.2e}")

    print(f"\nPosterior Distribution ΠЪ(q|П…) Summary:")
    print(f"{'='*60}")
    print(f"O¦:")
    print(f"  Mean = {np.mean(samples[:, 0]):.6f} W/cmBI")
    print(f"  Std  = {np.std(samples[:, 0]):.6f} W/cmBI")
    print(f"  95% CI = [{np.percentile(samples[:, 0], 2.5):.6f}, {np.percentile(samples[:, 0], 97.5):.6f}]")

    print(f"\nh:")
    print(f"  Mean = {np.mean(samples[:, 1]):.6e} W/cmBI/B°C")
    print(f"  Std  = {np.std(samples[:, 1]):.6e} W/cmBI/B°C")
    print(f"  95% CI = [{np.percentile(samples[:, 1], 2.5):.6e}, {np.percentile(samples[:, 1], 97.5):.6e}]")


    fig, axes = plt.subplots(2, 2, figsize=(14, 12))

    # Plot 1: Phi histogram (top left)
    axes[0, 0].hist(samples[:, 0], bins=30, density=True, alpha=0.7, color='skyblue', edgecolor='black')
    axes[0, 0].axvline(q_map[0], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[0]:.4f}')
    axes[0, 0].set_xlabel('O¦ (W/cmBI)')
    axes[0, 0].set_ylabel('Density')
    axes[0, 0].set_title('Marginal Posterior ΠЪ(O¦|П…)')
    axes[0, 0].legend()
    axes[0, 0].grid(True, alpha=0.3)

    # Plot 2: h histogram (top right)
    axes[0, 1].hist(samples[:, 1], bins=30, density=True, alpha=0.7, color='lightcoral', edgecolor='black')
    axes[0, 1].axvline(q_map[1], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[1]:.6f}')
```

```
axes[0, 1].set_xlabel('h (W/cmBI/B°C)')
axes[0, 1].set_ylabel('Density')
axes[0, 1].set_title('Marginal Posterior ПЂ(h|П…)')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Plot 3: Joint posterior scatter (bottom left)
axes[1, 0].scatter(samples[:, 0], samples[:, 1], alpha=0.5, c='purple', s=10)
axes[1, 0].plot(samples[:, 0], samples[:, 1], alpha=0.5, c='purple')
axes[1, 0].scatter(q_map[0], q_map[1], color='Magenta', s=200, marker='*', label='MAP')
axes[1, 0].set_xlabel('O¦ (W/cmBI)')
axes[1, 0].set_ylabel('h (W/cmBI/B°C)')
axes[1, 0].set_title('Joint Posterior Samples ПЂ(O¦,h|П…)')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Plot 4: Model fit with MAP parameters (bottom right)
rod.Phi_val = q_map[0]
rod.h_val = q_map[1]
T_map = rod.Ts(x_obs)

axes[1, 1].plot(x_obs, T_obs, 'ko', markersize=8, label='Observations')
axes[1, 1].plot(x_obs, T_map, 'r-', linewidth=2, label='MAP prediction')
axes[1, 1].set_xlabel('Position x (cm)')
axes[1, 1].set_ylabel('Temperature (B°C)')
axes[1, 1].set_title('Model Fit with MAP Parameters')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('problem1.1_results.png', dpi=150)
plt.show()
return samples, q_map, posterior_values, acceptance_rate
```

```python
from mcmc import *
def adaptive_metropolis_mcmc(q0, J_func, r_calc, M, rng_seed=None):
    """
    Metropolis-Hastings MCMC with Adaptive Metropolis (AM)

    Parameters:
    q0: initial sample
    J_func: initial proposal distribution function (for first 100 iterations)
    r_calc: function to compute acceptance ratio r(q*, q_{k-1})
    M: number of samples to generate
    rng_seed: random seed for reproducibility

    Returns:
    samples: array of MCMC samples
    acceptance_rate: acceptance rate of the chain
    Vk_at_101: the covariance matrix computed at k=101 (for problem 2.1)
    """
    if rng_seed is not None:
        np.random.seed(rng_seed)

    p = len(q0)  # dimension of parameter space (should be 2)
    q_current = np.array(q0, dtype=float)
    samples = np.zeros((M, p))
    samples[0] = q_current

    # AM parameters
    sp = 2.38**2 / p  # scaling factor (2.38^2/p)
    k0 = 100  # start adapting after 100 samples

    accepted = 0
    Vk_at_101 = None  # to store the covariance at k=101 for problem 2.1

    print("Running Adaptive Metropolis MCMC...")
    for i in range(1, M):
        # Step 2.1: Generate proposal
        if i < k0:
            # First k0 iterations: use fixed proposal J_func
            q_proposed = J_func(q_current)
        else:
            # Adaptive Metropolis proposal
            if i == k0:
                # First time using AM (k=101)
                # Compute covariance from samples[0:k0] (first 100 samples)
                samples_so_far = samples[:k0]
                Vk = np.cov(samples_so_far, rowvar=False)
                Vk_at_101 = Vk.copy()  # Save for problem 2.1
                print(f"\nAt k=101, computed Vk from first {k0} samples:")
                print(f"Vk = {Vk}")

                # Use this Vk for the proposal at k=101
                proposal_cov = sp * Vk
            elif np.mod(i, k0)==1:
                # For subsequent iterations, compute covariance from all samples up to i
                samples_so_far = samples[:i]
                Vk = np.cov(samples_so_far, rowvar=False)
                proposal_cov = sp * Vk

            # Generate proposal from N(q_current, sp * Vk)
            q_proposed = np.random.multivariate_normal(q_current, proposal_cov)

        # Step 2.2: Compute the ratio r(q*, q_{k-1})
        r = r_calc(q_proposed, q_current)

        # Step 2.3: Accept with probability min(1, r)
        if np.random.random() < min(1, r):
            q_current = q_proposed
            accepted += 1

        samples[i] = q_current

        if (i + 1) % 100 == 0:
            print(f"  Completed {i + 1}/{M} iterations, acceptance rate: {accepted/(i):.3f}")

    acceptance_rate = accepted / (M - 1)

    return samples, acceptance_rate, Vk_at_101, Vk

def run_adaptive_metropolis_for_heat_equation(M=None):
    """
    Run Adaptive Metropolis algorithm for the heat equation problem (Problem 2.1)
    """
    # Setup problem
    rod, x_obs, T_obs, sigma2_0, q0 = setup_heat_equation_problem('HW06_Problem1.mat')

    # Initial proposal covariance (same as in Problem 1.1)
    D = np.array([[1e-2, 0],
                  [0, 2e-10]])

    # Create initial proposal function J(q* | q_{k-1}) = N(q_{k-1}, D)
    # This will be used for the first 100 iterations
    def initial_proposal_func(q_current):
        return np.random.multivariate_normal(q_current, D)

    # Create function to compute r(q*, q_{k-1})
    posterior_ratio_func = create_posterior_ratio_func(rod, x_obs, T_obs, sigma2_0)

    if M is None:
        M = 1000

    print(f"\nRunning Adaptive Metropolis algorithm for {M} iterations...")
    print(f"Initial proposal distribution (first {100} iterations): N(q_prev, D) with D = diag([{D[0,0]}, {D[1,1]}])")
    print(f"AM parameters: sp = 2.38^2/{len(q0)} = {2.38**2/len(q0):.4f}, k0 = 100")

    samples, acceptance_rate, Vk_at_101, Vk_at_end = adaptive_metropolis_mcmc(
        q0,
        initial_proposal_func,
        posterior_ratio_func,
        M,
        rng_seed=42
    )

    print(f"\nFinal acceptance rate: {acceptance_rate:.3f}")

    # Problem 2.1: Report Vk at k=101
    print(f"\n{'='*60}")
    print("PROBLEM 2.1 RESULTS")
    print(f"{'='*60}")
    print(f"\nAt k=101, the new Vk generated from the first 100 samples is:")
    print(f"Vk = {Vk_at_101}")

    # Problem 2.2: Report Vk at final k
    print(f"\n{'='*60}")
    print("PROBLEM 2.2 RESULTS")
```

```
print(f"{'='*60}")
print(f"\nAt k=M, the final Vk generated is:")
print(f"Vk = {Vk_at_end}")


print("\nComputing posterior values for MAP estimate...")

# Prior parameters
prior_Phi_mean = -15
prior_Phi_std = 10
prior_h_mean = 0.001
prior_h_std = 0.005

# Compute unnormalized posterior for each sample
posterior_values = np.zeros(M)
for i, q in enumerate(samples):
    Phi, h = q
    rod.Phi_val = Phi
    rod.h_val = h
    T_model = rod.Ts(x_obs)

    # Likelihood
    n = len(x_obs)
    likelihood = np.exp(-0.5 * np.sum((T_obs - T_model)**2 / sigma2_0))

    # Prior
    prior = (norm.pdf(Phi, prior_Phi_mean, prior_Phi_std) *
             norm.pdf(h, prior_h_mean, prior_h_std))

    # Unnormalized posterior (pi(data|q)pi0(q))
    posterior_values[i] = likelihood * prior

# Find MAP estimate (maximum unnormalized posterior)
map_idx = np.argmax(posterior_values)
q_map = samples[map_idx]
post_map = posterior_values[map_idx]

print(f"\n{'='*60}")
print("PROBLEM 1.1 RESULTS")
print(f"{'='*60}")
print(f"\nMAP Estimate (q_MAP):")
print(f"  O¦ = {q_map[0]:.6f} W/cmBI")
print(f"  h = {q_map[1]:.6f} W/cmBI/B°C")
print(f"  ПЂ(П…|q_MAP)ПЂ0(q_MAP) = {post_map:.2e}")

print(f"\nPosterior Distribution ПЂ(q|П…) Summary:")
print(f"{'='*60}")
print(f"O¦:")
print(f"  Mean = {np.mean(samples[:, 0]):.6f} W/cmBI")
print(f"  Std  = {np.std(samples[:, 0]):.6f} W/cmBI")
print(f"  95% CI = [{np.percentile(samples[:, 0], 2.5):.6f}, {np.percentile(samples[:, 0], 97.5):.6f}]")

print(f"\nh:")
print(f"  Mean = {np.mean(samples[:, 1]):.6e} W/cmBI/B°C")
print(f"  Std  = {np.std(samples[:, 1]):.6e} W/cmBI/B°C")
print(f"  95% CI = [{np.percentile(samples[:, 1], 2.5):.6e}, {np.percentile(samples[:, 1], 97.5):.6e}]")

# Create diagnostic plots - 2x2 grid
fig, axes = plt.subplots(2, 2, figsize=(14, 12))

# Plot 1: Phi histogram (top left)
axes[0, 0].hist(samples[:, 0], bins=30, density=True, alpha=0.7, color='skyblue', edgecolor='black')
axes[0, 0].axvline(q_map[0], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[0]:.4f}')
axes[0, 0].set_xlabel('O¦ (W/cmBI)')
axes[0, 0].set_ylabel('Density')
axes[0, 0].set_title('Marginal Posterior ПЂ(O¦|П…)')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Plot 2: h histogram (top right)
axes[0, 1].hist(samples[:, 1], bins=30, density=True, alpha=0.7, color='lightcoral', edgecolor='black')
axes[0, 1].axvline(q_map[1], color='red', linestyle='--', linewidth=2, label=f'MAP: {q_map[1]:.6f}')
axes[0, 1].set_xlabel('h (W/cmBI/B°C)')
axes[0, 1].set_ylabel('Density')
axes[0, 1].set_title('Marginal Posterior ПЂ(h|П…)')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Plot 3: Joint posterior scatter (bottom left)
axes[1, 0].scatter(samples[:, 0], samples[:, 1], alpha=0.5, c='purple', s=10)
axes[1, 0].plot(samples[:, 0], samples[:, 1], alpha=0.5, c='purple')
axes[1, 0].scatter(q_map[0], q_map[1], color='Magenta', s=200, marker='*', label='MAP')
axes[1, 0].set_xlabel('O¦ (W/cmBI)')
axes[1, 0].set_ylabel('h (W/cmBI/B°C)')
axes[1, 0].set_title('Joint Posterior Samples ПЂ(O¦,h|П…)')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Plot 4: Model fit with MAP parameters (bottom right)
rod.Phi_val = q_map[0]
rod.h_val = q_map[1]
T_map = rod.Ts(x_obs)

axes[1, 1].plot(x_obs, T_obs, 'ko', markersize=8, label='Observations')
axes[1, 1].plot(x_obs, T_map, 'r-', linewidth=2, label='MAP prediction')
axes[1, 1].set_xlabel('Position x (cm)')
axes[1, 1].set_ylabel('Temperature (B°C)')
axes[1, 1].set_title('Model Fit with MAP Parameters')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('problem2.1_results.png', dpi=150)
plt.show()

return samples, posterior_values, q_map, Vk_at_101, acceptance_rate
```

```
from mcmc import *

def MetropolisHastingsGibbs(q0, s2_0, rod, x_obs, T_obs, D, M, ns, sigma2_s, rng_seed=None):
    """
    This function alternates between:
    1. Metropolis-Hastings step for q (O¦, h) given current sigma_squared
    2. Gibbs step for sigma_squared given current q

    Parameters:
    q0: initial q sample [Phi, h]
    s2_0: initial sigma_squared value
    rod: heat rod object for model evaluation
    x_obs: observation locations
    T_obs: observed temperatures
    D: proposal covariance matrix for q
    M: number of samples to generate
    ns, sigma2_s: hyperparameters for inverse gamma prior on sigma_squared
    rng_seed: random seed for reproducibility

    Returns:
    q_samples: array of q samples (M x 2)
    s2_samples: array of sigma_squared samples (M)
    acceptance_rate: acceptance rate of the chain for q
    """
    if rng_seed is not None:
        np.random.seed(rng_seed)

    q_current = np.array(q0, dtype=float)
    s2_current = s2_0
    n_obs = len(x_obs)

    q_samples = np.zeros((M, len(q0)))
    s2_samples = np.zeros(M)

    q_samples[0] = q_current
    s2_samples[0] = s2_current

    # Prior parameters
    prior_Phi_mean = -15
    prior_Phi_std = 10
    prior_h_mean = 0.001
    prior_h_std = 0.005

    accepted = 0

    print("Running Metropolis-Hastings with Gibbs sampling for unknown sigma_squared...")
    print(f"ns = {ns}, sigma2_s = {sigma2_s}")
    print(f"Initial sigma_squared = {s2_0}")

    for k in range(1, M):
        # ============================================================
        # STEP 2.1: METROPOLIS-HASTINGS FOR q GIVEN CURRENT sigma^2
        # ============================================================

        # proposal from N(q_current, D)
        q_proposed = np.random.multivariate_normal(q_current, D)

        # Compute acceptance ratio r(q*, q_{k-1}) using current sigma_squared
        # r = [pi(data|q*,sigma_squared)pi0(q*)] / [pi(data|q_{k-1},sigma_squared)pi0(q_{k-1})] (symmetric J(q*|q_k-1))

        # Evaluate likelihood for proposed q
        Phi_star, h_star = q_proposed
        rod.Phi_val = Phi_star
        rod.h_val = h_star
        T_model_star = rod.Ts(x_obs)
        log_likelihood_star = -0.5 * np.sum((T_obs - T_model_star)**2 / s2_current)

        # Evaluate likelihood for current q
        Phi_curr, h_curr = q_current
        rod.Phi_val = Phi_curr
        rod.h_val = h_curr
        T_model_curr = rod.Ts(x_obs)
        log_likelihood_curr = -0.5 * np.sum((T_obs - T_model_curr)**2 / s2_current)

        # Log priors for q
        log_prior_star = (norm.logpdf(Phi_star, prior_Phi_mean, prior_Phi_std) +
                          norm.logpdf(h_star, prior_h_mean, prior_h_std))
        log_prior_curr = (norm.logpdf(Phi_curr, prior_Phi_mean, prior_Phi_std) +
                          norm.logpdf(h_curr, prior_h_mean, prior_h_std))

        # Log ratio (proposal is symmetric, so it cancels out)
        log_ratio = (log_likelihood_star + log_prior_star) - (log_likelihood_curr + log_prior_curr)
        r = np.exp(log_ratio)

        # Accept or reject
        if np.random.random() < min(1, r):
            q_current = q_proposed
            accepted += 1

        q_samples[k] = q_current

        # ============================================================
        # STEP 2.2: GIBBS STEP FOR sigma_squared GIVEN CURRENT q
        # ============================================================

        # Compute sum of squared errors for current q
        Phi, h = q_current
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)
        SSq = np.sum((T_obs - T_model)**2)

        # Inverse Gamma parameters for Gibbs sampling
        # From textbook p. 163: p(sigma_squared|data,q) = Inv-gamma(O± + n/2, OI + SSq/2)
        # where O± = ns/2, OI = (ns * sigma_squared_s)/2
        aval = ns/2 + n_obs/2
        bval = (ns * sigma2_s)/2 + SSq/2

        # Sample from Inv-Gamma(aval, bval)
        # Using the relationship:
        # If X ~ Gamma(shape=aval, rate=bval), then 1/X ~ Inv-Gamma(aval, bval)
        # numpy.random.gamma requires shape and scale (1/rate)
        gamma_sample = np.random.gamma(aval, 1/bval)
        s2_current = 1 / gamma_sample

        s2_samples[k] = s2_current

        # Progress reporting
        if (k + 1) % 1000 == 0:
            print(f"  Completed {k + 1}/{M} iterations, MCMC acceptance rate: {accepted/(k):.3f}, sigma_squared = {s2_current:.4f}")

    acceptance_rate = accepted / (M - 1)
```

```
    return q_samples, s2_samples, acceptance_rate


def run_metropolis_gibbs_for_heat_equation(M=10000,D = None, data_name=None, ns=0.01, sigma2_s=4.0, burnin=300):
    """
    Run Metropolis-Hastings with Gibbs sampling for unknown sigma_squared

    Parameters:
    M: number of samples (default 10000)
    data_name: MATLAB data file name
    ns, sigma2_s: hyperparameters for inverse gamma prior on sigma_squared
    burnin: number of burn-in samples to remove
    """
    if data_name is None:
        data_name = 'HW06_Problem3.mat'

    rod, x_obs, T_obs, sigma2_0, q0 = setup_heat_equation_problem(data_name)

    # Proposal covariance matrix for q
    if D is None:
        D = np.array([[1e-2, 0],
                      [0, 2e-10]])

    # Initial sigma_squared (use the known value from problem 1.1 as starting point)
    s2_0 = sigma2_0

    print("\n" + "="*70)
    print("PROBLEM 1.2: METROPOLIS-HASTINGS WITH GIBBS FOR UNKNOWN sigma_squared")
    print("="*70)
    print(f"Hyperparameters: ns = {ns}, sigma_squared_s = {sigma2_s}")
    print(f"Initial sigma_squared = {s2_0}")
    print(f"Running for M = {M} iterations with {burnin} burn-in samples")

    # MCMC with Gibbs
    q_samples, s2_samples, acceptance_rate = MetropolisHastingsGibbs(
        q0, s2_0, rod, x_obs, T_obs, D, M, ns, sigma2_s, rng_seed=42
    )

    print(f"\nFinal acceptance rate for q: {acceptance_rate:.3f}")

    # Remove burn-in samples
    q_samples_post = q_samples[burnin:]
    s2_samples_post = s2_samples[burnin:]

    # Prior parameters for q (for MAP calculation)
    prior_Phi_mean = -15
    prior_Phi_std = 10
    prior_h_mean = 0.001
    prior_h_std = 0.005

    # Find MAP estimate (maximum unnormalized posterior with sigma_squared)
    print("\nComputing posterior values for MAP estimate...")

    posterior_values = np.zeros(len(q_samples_post))
    for i, (q, s2) in enumerate(zip(q_samples_post, s2_samples_post)):
        Phi, h = q
        rod.Phi_val = Phi
        rod.h_val = h
        T_model = rod.Ts(x_obs)

        # Likelihood with current sigma_squared
        n = len(x_obs)
        likelihood = (1/np.sqrt(2*np.pi*s2))**n * np.exp(-0.5 * np.sum((T_obs - T_model)**2 / s2))

        # Prior for q
        prior_q = (norm.pdf(Phi, prior_Phi_mean, prior_Phi_std) *
                   norm.pdf(h, prior_h_mean, prior_h_std))

        # Prior for sigma_squared (Inverse Gamma)
        # p(sigma_squared) \propto (sigma_squared)^(-(ns/2+1)) * exp(-(ns*sigma_squared_s)/(2sigma_squared))
        log_prior_sigma = -(ns/2 + 1) * np.log(s2) - (ns * sigma2_s) / (2 * s2)
        prior_sigma = np.exp(log_prior_sigma)

        # Unnormalized posterior
        posterior_values[i] = likelihood * prior_q * prior_sigma

map_idx = np.argmax(posterior_values)
q_map = q_samples_post[map_idx]
s2_map = s2_samples_post[map_idx]
post_map = posterior_values[map_idx]

print(f"\n{'='*60}")
print("PROBLEM 1.2 RESULTS (with unknown sigma_squared)")
print(f"{'='*60}")
print(f"\nMAP Estimate (q_MAP, sigma_squared_MAP):")
print(f"  O¦ = {q_map[0]:.6f} W/cmBI")
print(f"  h = {q_map[1]:.6f} W/cmBI/B°C")
print(f"  sigma_squared = {s2_map:.6f} (B°C)BI")
print(f"  sigma = {np.sqrt(s2_map):.4f} B°C")
print(f"  pi(data|q_MAP,sigma_squared_MAP)pi0(q_MAP,sigma_squared_MAP) = {post_map:.2e}")

print(f"\nPosterior Distribution pi(q,sigma_squared|data) Summary (after burn-in):")
print(f"{'='*60}")
print(f"O¦:")
print(f"  Mean = {np.mean(q_samples_post[:, 0]):.6f} W/cmBI")
print(f"  Std  = {np.std(q_samples_post[:, 0]):.6f} W/cmBI")
print(f"  95% CI = [{np.percentile(q_samples_post[:, 0], 2.5):.6f}, "
      f"{np.percentile(q_samples_post[:, 0], 97.5):.6f}]")

print(f"\nh:")
print(f"  Mean = {np.mean(q_samples_post[:, 1]):.6e} W/cmBI/B°C")
print(f"  Std  = {np.std(q_samples_post[:, 1]):.6e} W/cmBI/B°C")
print(f"  95% CI = [{np.percentile(q_samples_post[:, 1], 2.5):.6e}, "
      f"{np.percentile(q_samples_post[:, 1], 97.5):.6e}]")

print(f"\nsigma_squared:")
print(f"  Mean = {np.mean(s2_samples_post):.6f} (B°C)BI")
print(f"  Std  = {np.std(s2_samples_post):.6f} (B°C)BI")
print(f"  95% CI = [{np.percentile(s2_samples_post, 2.5):.6f}, "
      f"{np.percentile(s2_samples_post, 97.5):.6f}]")

# Create diagnostic plots
fig, axes = plt.subplots(2, 3, figsize=(18, 12))

# Plot 1: Phi histogram
axes[0, 0].hist(q_samples_post[:, 0], bins=30, density=True, alpha=0.7,
                color='skyblue', edgecolor='black')
axes[0, 0].axvline(q_map[0], color='red', linestyle='--', linewidth=2,
                   label=f'MAP: {q_map[0]:.4f}')
axes[0, 0].set_xlabel('O¦ (W/cmBI)')
axes[0, 0].set_ylabel('Density')
axes[0, 0].set_title('Marginal Posterior pi(O¦|data)')
axes[0, 0].legend()
```

```
axes[0, 0].grid(True, alpha=0.3)

# Plot 2: h histogram
axes[0, 1].hist(q_samples_post[:, 1], bins=30, density=True, alpha=0.7,
                color='lightcoral', edgecolor='black')
axes[0, 1].axvline(q_map[1], color='red', linestyle='--', linewidth=2,
                   label=f'MAP: {q_map[1]:.6f}')
axes[0, 1].set_xlabel('h (W/cmBI/B°C)')
axes[0, 1].set_ylabel('Density')
axes[0, 1].set_title('Marginal Posterior pi(h|data)')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# Plot 3: sigma_squared histogram with KDE
axes[0, 2].hist(s2_samples_post, bins=30, density=True, alpha=0.7,
                color='lightgreen', edgecolor='black', label='Histogram')

# Add KDE
from scipy.stats import gaussian_kde
kde = gaussian_kde(s2_samples_post)
x_range = np.linspace(s2_samples_post.min(), s2_samples_post.max(), 200)
axes[0, 2].plot(x_range, kde(x_range), 'b-', linewidth=2, label='KDE')
axes[0, 2].axvline(s2_map, color='red', linestyle='--', linewidth=2,
                   label=f'MAP: {s2_map:.4f}')
axes[0, 2].set_xlabel('sigma_squared (B°C)BI')
axes[0, 2].set_ylabel('Density')
axes[0, 2].set_title('Marginal Posterior pi(sigma_squared|data)')
axes[0, 2].legend()
axes[0, 2].grid(True, alpha=0.3)

# Plot 4: Joint posterior scatter (Phi vs h) colored by sigma_squared
scatter = axes[1, 0].scatter(q_samples_post[:, 0], q_samples_post[:, 1],
                             alpha=0.5, c=s2_samples_post, cmap='viridis', s=10)
axes[1, 0].scatter(q_map[0], q_map[1], color='red', s=200, marker='*', label='MAP')
axes[1, 0].set_xlabel('O¦ (W/cmBI)')
axes[1, 0].set_ylabel('h (W/cmBI/B°C)')
axes[1, 0].set_title('Joint Posterior Samples (colored by sigma_squared)')
plt.colorbar(scatter, ax=axes[1, 0], label='sigma_squared (B°C)BI')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# Plot 5: Model fit with MAP parameters
rod.Phi_val = q_map[0]
rod.h_val = q_map[1]
T_map = rod.Ts(x_obs)

axes[1, 1].plot(x_obs, T_obs, 'ko', markersize=8, label='Observations')
axes[1, 1].plot(x_obs, T_map, 'r-', linewidth=2, label='MAP prediction')
axes[1, 1].fill_between(x_obs, T_map - 2*np.sqrt(s2_map), T_map + 2*np.sqrt(s2_map),
                        color='red', alpha=0.2, label=f'B±2Πή (Πή={np.sqrt(s2_map):.2f}B°C)')
axes[1, 1].set_xlabel('Position x (cm)')
axes[1, 1].set_ylabel('Temperature (B°C)')
axes[1, 1].set_title('Model Fit with MAP Parameters')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

# Plot 6: Trace plot of sigma_squared
axes[1, 2].plot(range(M), s2_samples, 'g-', alpha=0.5, linewidth=0.5)
axes[1, 2].axhline(s2_map, color='red', linestyle='--', label=f'MAP: {s2_map:.4f}')
axes[1, 2].axvline(burnin, color='black', linestyle=':', label=f'Burn-in ({burnin})')
axes[1, 2].set_xlabel('Iteration')
axes[1, 2].set_ylabel('sigma_squared (B°C)BI')
axes[1, 2].set_title('Trace Plot of sigma_squared Samples')
axes[1, 2].legend()
axes[1, 2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('problem1.2_results.png', dpi=150)
plt.show()

return q_samples, s2_samples, q_map, s2_map, posterior_values, acceptance_rate
```

```python
import sympy as sp
import numpy as np

class SteadyStateRod:
    def __init__(self, a, b, h_val, k_val, Phi_val, Tamb_val, L):
        """
        Initialize the rod parameters.
        a, b: cross-section dimensions (cm)
        h_val: convective heat transfer coefficient (W/cm^2 C)
        k_val: thermal conductivity (W/cm C)
        Phi_val: source heat flux (W/cm^2)
        Tamb_val: ambient temperature (C)
        """
        # Store numeric values
        self.a_val = a
        self.b_val = b
        self.h_val = h_val
        self.k_val = k_val
        self.Phi_val = Phi_val
        self.Tamb_val = Tamb_val
        self.L = L

        # Define symbolic variables
        x, h, k, Phi, Tamb = sp.symbols('x h k Phi Tamb')
        self.x = x
        self.h = h
        self.k = k
        self.Phi = Phi
        self.Tamb = Tamb

        # Precompute gamma
        self.gamma = sp.sqrt(2*(a+b)*h/(a*b*k))

        # Define c1, c2
        c1 = -Phi/(k*self.gamma) * ((sp.exp(self.gamma*self.L)*(h + k*self.gamma)) / (sp.exp(-self.gamma*self.L)*(h - k*self.gamma) \
                                                                                       + sp.exp(self.gamma*self.L)*(h + k*self.gamma)))
        c2 = Phi/(k*self.gamma) + c1
        self.c1 = c1
        self.c2 = c2

        # Ts(x) symbolic
        Ts_expr = c1*sp.exp(-self.gamma*x) + c2*sp.exp(self.gamma*x) + Tamb
        self.Ts_expr = Ts_expr

        # Precompute derivatives symbolically
        self.dTs_dh_expr = sp.diff(Ts_expr, h)
        self.dTs_dk_expr = sp.diff(Ts_expr, k)
        self.dTs_dPhi_expr = sp.diff(Ts_expr, Phi)


        # Lambdify for fast numeric evaluation
        vars = (x, h, k, Phi, Tamb)
        self.Ts_func = sp.lambdify(vars, Ts_expr, 'numpy')
        self.dTs_dh_func = sp.lambdify(vars, self.dTs_dh_expr, 'numpy')
        self.dTs_dk_func = sp.lambdify(vars, self.dTs_dk_expr, 'numpy')
        self.dTs_dPhi_func = sp.lambdify(vars, self.dTs_dPhi_expr, 'numpy')

    def Ts(self, x_vals, h=None, k=None, Phi=None, Tamb=None):
        """Evaluate Ts at given x values. Use class defaults if not provided."""
        h = h if h is not None else self.h_val
        k = k if k is not None else self.k_val
        Phi = Phi if Phi is not None else self.Phi_val
        Tamb = Tamb if Tamb is not None else self.Tamb_val
        return self.Ts_func(x_vals, h, k, Phi, Tamb)

    def dTs_dh(self, x_vals, h=None, k=None, Phi=None, Tamb=None):
        h = h if h is not None else self.h_val
        k = k if k is not None else self.k_val
        Phi = Phi if Phi is not None else self.Phi_val
        Tamb = Tamb if Tamb is not None else self.Tamb_val
        return self.dTs_dh_func(x_vals, h, k, Phi, Tamb)


    def dTs_dk(self, x_vals, h=None, k=None, Phi=None, Tamb=None):
        h = h if h is not None else self.h_val
        k = k if k is not None else self.k_val
        Phi = Phi if Phi is not None else self.Phi_val
        Tamb = Tamb if Tamb is not None else self.Tamb_val
        return self.dTs_dk_func(x_vals, h, k, Phi, Tamb)

    def dTs_dPhi(self, x_vals, h=None, k=None, Phi=None, Tamb=None):
        h = h if h is not None else self.h_val
        k = k if k is not None else self.k_val
        Phi = Phi if Phi is not None else self.Phi_val
        Tamb = Tamb if Tamb is not None else self.Tamb_val
        return self.dTs_dPhi_func(x_vals, h, k, Phi, Tamb)
```